copia_01_profiling_empresas_profesional_con_matching - copia

April 12, 2026

1 Data Profiling e Integración de Fuentes para Clustering de Empresas

1.1 Objetivo

Este cuaderno documenta el proceso de análisis, validación e integración de dos fuentes de datos empresariales con el objetivo de construir un dataset adecuado para técnicas de clustering.

1.2 Fuentes de datos

- Dataset 1: Leads provenientes de CRM
- Dataset 2: Registro de horas trabajadas por empresa

1.3 Enfoque

Se sigue una metodología basada en:

1. Data profiling
2. Validación de hipótesis de integración
3. Evaluación de consistencia entre fuentes
4. Toma de decisiones basada en evidencia
5. Recomendación de siguiente paso (staging y estrategia de integración)

1.4 1. Imports y configuración

[20]:
```
import pandas as pd
import numpy as np
import re
import unicodedata
from pathlib import Path

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

1.5 2. Configuración de archivos

[21]:
```
LEADS_FILE = 'leads.xlsx'
HORAS_FILE = 'proyectos_empresa.xlsx'

N_FILAS_PRUEBA = None
```

1.6 3. Función de carga de datos

[22]:
```
def leer_archivo(path, nrows=None):
    path = Path(path)
    suffix = path.suffix.lower()

    if suffix == '.csv':
        return pd.read_csv(path, nrows=nrows)
    elif suffix in ['.xlsx', '.xls']:
        return pd.read_excel(path, nrows=nrows)
    else:
        raise ValueError(f'Formato no soportado: {suffix}')
```

1.7 4. Carga de datasets

[23]:
```
df_leads = leer_archivo(LEADS_FILE, nrows=N_FILAS_PRUEBA)
df_horas = leer_archivo(HORAS_FILE, nrows=N_FILAS_PRUEBA)

print('Leads:', df_leads.shape)
print('Horas:', df_horas.shape)
```

```
Leads: (440, 15)
Horas: (412, 18)
```

```
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
```

1.8 4.b Perfilado rápido (calidad y esquema)

[24]:
```
def _profile_table(df: pd.DataFrame) -> pd.DataFrame:
    out = pd.DataFrame({
        'columna': df.columns,
        'dtype': [str(df[c].dtype) for c in df.columns],
        'n_null': [int(df[c].isna().sum()) for c in df.columns],
        'pct_null': [float(df[c].isna().mean() * 100) for c in df.columns],
        'n_unique': [int(df[c].nunique(dropna=True)) for c in df.columns],
    })
    return out.sort_values(['pct_null', 'n_unique'], ascending=[False, True]).
 ↪reset_index(drop=True)
```

```
def _normalize_for_compare(value) -> str:
    if value is None:
        return ''
    if isinstance(value, float) and np.isnan(value):
        return ''
    s = str(value).strip()
    if not s:
        return ''
    s = unicodedata.normalize('NFKD', s)
    s = ''.join(ch for ch in s if not unicodedata.combining(ch))
    s = s.upper()
    s = re.sub(r'[^A-Z0-9 ]+', ' ', s)
    s = re.sub(r'\s+', ' ', s).strip()
    return s

def _company_quality(df: pd.DataFrame, col: str, label: str) -> None:
    if col not in df.columns:
        print(f'[{label}] Columna no encontrada: {col}')
        return
    s = df[col]
    s_str = s.astype('string')
    blank = s_str.isna() | (s_str.str.strip() == '')
    print(f'\n[{label}] Calidad de {col}:')
    print('- filas:', len(df))
    print('- n_null:', int(s.isna().sum()))
    print('- n_blank (null o vacío):', int(blank.sum()))
    print('- n_unique (no-null):', int(s.nunique(dropna=True)))
    print('- n_duplicadas (por valor no-null):', int(s.dropna().duplicated().
 ↪sum()))

    top = (s_str.fillna('')
              .map(lambda x: x.strip())
              .replace('', pd.NA)
              .dropna()
              .value_counts()
              .head(15)
              .reset_index())
    if len(top):
        top.columns = [col, 'freq']
        display(top)

    # Normalización rápida para estimar colisiones
    norm = s_str.map(_normalize_for_compare)
    norm = norm.replace('', pd.NA).dropna()
    if len(norm):
        print('- n_unique_norm:', int(norm.nunique()))
```

```
        print('- colisiones_por_norm (valores distintos que caen al mismo norm):
 ↪', int(norm.duplicated().sum()))

def _exact_match_summary() -> None:
    if 'Company' not in df_leads.columns or 'EMPRESA' not in df_horas.columns:
        print('\n[JOIN] No se puede calcular resumen: falta Company o EMPRESA.')
        return

    leads_raw = (df_leads['Company'].astype('string').fillna('').map(lambda x:␣
 ↪x.strip()))
    horas_raw = (df_horas['EMPRESA'].astype('string').fillna('').map(lambda x:␣
 ↪x.strip()))
    leads_raw = leads_raw.replace('', pd.NA).dropna()
    horas_raw = horas_raw.replace('', pd.NA).dropna()

    raw_matches = sorted(set(leads_raw).intersection(set(horas_raw)))
    print('\n[JOIN] Coincidencias exactas (sin normalizar):', len(raw_matches))
    if len(raw_matches):
        print('  Muestra:', raw_matches[:20])

    leads_norm = leads_raw.map(_normalize_for_compare).replace('', pd.NA).
 ↪dropna()
    horas_norm = horas_raw.map(_normalize_for_compare).replace('', pd.NA).
 ↪dropna()
    norm_matches = sorted(set(leads_norm).intersection(set(horas_norm)))
    print('[JOIN] Coincidencias exactas (normalizadas):', len(norm_matches))
    if len(norm_matches):
        print('  Muestra:', norm_matches[:20])

print('--- Perfilado LEADS ---')
display(_profile_table(df_leads))

print('\n--- Perfilado HORAS ---')
display(_profile_table(df_horas))

# Calidad de las llaves candidatas para join
_company_quality(df_leads, 'Company', 'LEADS')
_company_quality(df_horas, 'EMPRESA', 'HORAS')

# Evidencia cuantitativa: ¿hay intersección exacta?
_exact_match_summary()
```

```
--- Perfilado LEADS ---
```

	columna	dtype	n_null	pct_null	n_unique
0	First Name	float64	440	100.000000	0
1	Last Name	float64	440	100.000000	0
2	Email	float64	440	100.000000	0

3	Phone	float64	440	100.000000	0
4	Mobile	float64	440	100.000000	0
5	Website	float64	440	100.000000	0
6	No. of Employees	float64	440	100.000000	0
7	Annual Revenue	float64	440	100.000000	0
8	Linkedin	float64	440	100.000000	0
9	País.	str	415	94.318182	8
10	Lead Source	str	408	92.727273	1
11	Industry	str	408	92.727273	4
12	Cargo	str	87	19.772727	290
13	Interés	str	0	0.000000	1
14	Company	str	0	0.000000	330

--- Perfilado HORAS ---

	columna	dtype	n_null	pct_null	n_unique
0	AVANCE_REAL	float64	412	100.000000	0
1	AVANCE_ESTIMADO	float64	412	100.000000	0
2	ID_COL_RESPONSABLE	float64	379	91.990291	7
3	FACTURACION	float64	235	57.038835	120
4	HORAS_ESTIMADAS	float64	187	45.388350	107
5	HORAS_EJECUTADAS_FACTURABLES	float64	26	6.310680	314
6	HORAS_EJECUTADAS	float64	20	4.854369	319
7	FECHA_CORTE	datetime64[us]	19	4.611650	5
8	EN_EJECUCION	float64	4	0.970874	2
9	MOSTRAR_LISTAS	int64	0	0.000000	2
10	LUGAR_POR_DEFECTO	int64	0	0.000000	2
11	TIPO_ALCANCE	str	0	0.000000	3
12	OCUPACION	str	0	0.000000	4
13	ID_ESP	int64	0	0.000000	15
14	EMPRESA	str	0	0.000000	120
15	ID_EMP	int64	0	0.000000	120
16	NOMBRE	str	0	0.000000	372
17	ID_PRO	int64	0	0.000000	412

[LEADS] Calidad de Company:
- filas: 440
- n_null: 0
- n_blank (null o vacío): 0
- n_unique (no-null): 330
- n_duplicadas (por valor no-null): 110

	Company	freq
0	GRUPO ALEN	10
1	PEPSICO	7
2	GRUPO BIMBO	7
3	WALMART	7
4	BACARDI	5

```
5                            LAMOSA     5
6                       CASA CUERVO     5
7   SEGUROS MONTERREY NEW YORK LIFE     5
8                          UNILEVER     4
9                               AIG     4
10                         NOVARTIS     4
11                       ASTRAZENECA     3
12                            SANOFI     3
13                             RAPPI     3
14                       GNP SEGUROS     3
```

```
- n_unique_norm: 328
- colisiones_por_norm (valores distintos que caen al mismo norm): 112

[HORAS] Calidad de EMPRESA:
- filas: 412
- n_null: 0
- n_blank (null o vacío): 0
- n_unique (no-null): 120
- n_duplicadas (por valor no-null): 292
```

```
                         EMPRESA  freq
0                Corporacion GPF    66
1             Corporación Maresa    22
2                            FPA    21
3                   Veolia Latam    21
4            Aseguradora del Sur    20
5                         Intaco    12
6                      La Fabril    10
7                   NOVA Ecuador    10
8                  Telefónica EC    10
9   Millicom - Telefónica PA, NI     9
10                         Adium     8
11         Seguros del Pichincha     7
12                  Veolia China     7
13                      Induglob     6
14              Veolia Argentina     6
```

```
- n_unique_norm: 119
- colisiones_por_norm (valores distintos que caen al mismo norm): 293

[JOIN] Coincidencias exactas (sin normalizar): 0
[JOIN] Coincidencias exactas (normalizadas): 0
```

1.9 5. Validación de hipótesis de integración

1.9.1 Hipótesis

Se plantea que:

Las empresas presentes en el dataset de leads deberían coincidir con las empresas presentes en el dataset de horas trabajadas.

1.9.2 Objetivo

Validar si es posible realizar un join confiable entre ambas fuentes.

1.10 6. Evaluación detallada (coincidencias exactas normalizadas y aproximadas)

El resumen de la sección **4.b (Perfilado rápido)** puede ampliarse con una rutina más detallada de normalización y similitud. Esta sección se conserva porque aporta evidencia técnica adicional:

- normaliza nombres con eliminación de acentos,
- reduce ruido por sufijos legales comunes,
- calcula coincidencias exactas después de normalizar,
- y genera sugerencias por similitud para evidenciar que no existen matches confiables.

Esto respalda formalmente la decisión de **no forzar un join** entre ambas fuentes usando solo el nombre de la empresa.

[25]:
```
import re
import unicodedata
import pandas as pd

def _strip_accents(s: str) -> str:
    return ''.join(ch for ch in unicodedata.normalize('NFKD', s) if not␣
 ↪unicodedata.combining(ch))

def _normalize_name(s: str) -> str:
    if s is None:
        return ""
    s = str(s).strip()
    s = _strip_accents(s)
    s = s.upper()
    # quitar caracteres raros, dejar letras/números/espacios
    s = re.sub(r"[^A-Z0-9 ]+", " ", s)
    # quitar sufijos legales comunes
    s = re.sub(r"\b(SA|S A|S\.A|S\.A\.
 ↪S|SAS|LTDA|CIA|CORP|INC|LLC|DE|DEL|LA|EL)\b", " ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s

# Tomar listas directamente desde los DataFrames (evita dependencia de␣
 ↪secciones eliminadas)
if 'Company' not in df_leads.columns or 'EMPRESA' not in df_horas.columns:
    print("No se puede comparar: falta Company o EMPRESA en los dataframes.")
else:
    companies = (df_leads['Company'].astype('string').fillna('')
```

```
                .map(lambda x: x.strip())
                .replace('', pd.NA)
                .dropna()
                .unique()
                .tolist())
    empresas = (df_horas['EMPRESA'].astype('string').fillna('')
                .map(lambda x: x.strip())
                .replace('', pd.NA)
                .dropna()
                .unique()
                .tolist())

    if not companies or not empresas:
        print("No hay valores suficientes para comparar (listas vacías).")
    else:
        comp_norm = {c: _normalize_name(c) for c in companies}
        emp_norm = {e: _normalize_name(e) for e in empresas}

        # 1) Coincidencias exactas después de normalizar
        inv_emp = {}
        for e, ne in emp_norm.items():
            inv_emp.setdefault(ne, []).append(e)

        exact_matches = []
        for c, nc in comp_norm.items():
            for e in inv_emp.get(nc, []):
                exact_matches.append({
                    "Company": c,
                    "EMPRESA": e,
                    "tipo": "exact_norm",
                    "score": 100
                })

        df_exact = pd.DataFrame(exact_matches)
        print(f"Coincidencias EXACTAS (tras normalizar): {len(df_exact)}")
        if len(df_exact):
            display(df_exact.sort_values(['Company', 'EMPRESA']).
↪reset_index(drop=True))

        # 2) Coincidencias por similitud (fuzzy). Si está rapidfuzz, mejor; si␣
↪no, usa difflib.
        rows = []
        try:
            from rapidfuzz import process, fuzz
            scorer = fuzz.token_set_ratio
            emp_choices = list(emp_norm.keys())
            for c, nc in comp_norm.items():
```

```
                best = process.extractOne(nc, emp_choices, scorer=scorer)
                if best is None:
                    continue
                best_norm, score, _ = best
                e_orig = inv_emp.get(best_norm, [None])[0]
                rows.append({
                    "Company": c,
                    "Company_norm": nc,
                    "EMPRESA_best": e_orig,
                    "EMPRESA_norm": best_norm,
                    "score": float(score),
                })
            engine = "rapidfuzz"
        except Exception:
            from difflib import SequenceMatcher

            def ratio(a, b):
                return SequenceMatcher(None, a, b).ratio() * 100

            emp_choices = list(emp_norm.keys())
            for c, nc in comp_norm.items():
                best_norm = None
                best_score = -1
                for en in emp_choices:
                    sc = ratio(nc, en)
                    if sc > best_score:
                        best_score = sc
                        best_norm = en
                e_orig = inv_emp.get(best_norm, [None])[0] if best_norm is not␣
↪None else None
                rows.append({
                    "Company": c,
                    "Company_norm": nc,
                    "EMPRESA_best": e_orig,
                    "EMPRESA_norm": best_norm,
                    "score": float(best_score),
                })
            engine = "difflib"

        df_fuzzy = pd.DataFrame(rows).sort_values('score', ascending=False).
↪reset_index(drop=True)
        print(f"\nMotor de similitud: {engine}")
        print("Sugerencia: revisar coincidencias con score alto (p.ej. >= 80).")

        display(df_fuzzy.head(30))
        umbral = 80
        df_high = df_fuzzy[df_fuzzy['score'] >= umbral]
```

```
        print(f"\nCoincidencias sugeridas con score >= {umbral}:␣
↪{len(df_high)}")
        display(df_high.reset_index(drop=True))
```

```
Coincidencias EXACTAS (tras normalizar): 0

Motor de similitud: rapidfuzz
Sugerencia: revisar coincidencias con score alto (p.ej. >= 80).
```

	Company	Company_norm	EMPRESA_best	EMPRESA_norm	score
0	OSRAM	OSRAM	CORSAM	CORSAM	72.727273
1	FEMSA	FEMSA	FADESA	FADESA	72.727273
2	SMI	SMI	BMI	BMI	66.666667
3	BIC	BIC	BAC	BAC	66.666667
4	UCB	UCB	UPC	UPC	66.666667
5	CBC	CBC	BAC	BAC	66.666667
6	Chronos	CHRONOS	CONSEP	CONSEP	61.538462
7	LACOSTE	LACOSTE	CONSEP	CONSEP	61.538462
8	BACARDI	BACARDI	BANRED	BANRED	61.538462
9	BACHOCO	BACHOCO	BAC	BAC	60.000000
10	PEPSICO	PEPSICO	SIC	SIC	60.000000
11	Abbvie	ABBVIE	AVIS	AVIS	60.000000
12	Onapis	ONAPIS	AVIS	AVIS	60.000000
13	BALL	BALL	BAC	BAC	57.142857
14	PROESSA	PROESSA	PROPHAR	PROPHAR	57.142857
15	PPG	PPG	KPMG	KPMG	57.142857
16	BECLE SAB	BECLE SAB	CEPSA	CEPSA	57.142857
17	AIG	AIG	AVIS	AVIS	57.142857
18	URBI	URBI	BMI	BMI	57.142857
19	Pozuelo	POZUELO	PUCE	PUCE	54.545455
20	ONTEX	ONTEX	CONSEP	CONSEP	54.545455
21	TRANE	TRANE	BANRED	BANRED	54.545455
22	ESACERO S.A.	ESACERO	ESPE	ESPE	54.545455
23	Parauco	PARAUCO	PUCE	PUCE	54.545455
24	Arcor	ARCOR	CORSAM	CORSAM	54.545455
25	COMEX	COMEX	CONSEP	CONSEP	54.545455
26	ICONN	ICONN	CONSEP	CONSEP	54.545455
27	Confiteca	CONFITECA	CONSEP	CONSEP	53.333333
28	CXO Forum	CXO FORUM	CORSAM	CORSAM	53.333333
29	EUROFARMA	EUROFARMA	PROPHAR	PROPHAR	50.000000

```
Coincidencias sugeridas con score >= 80: 0

Empty DataFrame
Columns: [Company, Company_norm, EMPRESA_best, EMPRESA_norm, score]
Index: []
```

1.10.1 Interpretación técnica de esta validación

Si esta sección produce:

- **0 coincidencias exactas tras normalización**, y
- **0 coincidencias con score alto**,

entonces existe evidencia suficiente para afirmar que ambas fuentes no pueden integrarse de forma confiable mediante el nombre de la empresa.

Este resultado no representa una falla del código, sino un hallazgo del análisis de calidad e integración de datos.

> Nota de organización: la generación de **data cruda normalizada** (staging) y el **export** a CSV se movieron a `02_data_cleaning/01_raw_normalizado_export.ipynb` para no mezclar ingesta/profiling con cleaning.

1.11 7. Resultado de la validación

No se encontraron coincidencias exactas entre las empresas de ambas fuentes.

1.11.1 Interpretación

Esto indica que:

- Las fuentes no están alineadas
- No es posible realizar un join directo confiable
- Existe inconsistencia en la representación de entidades

1.11.2 Conclusión

Se rechaza la hipótesis de integración directa.

1.12 8. Próximos pasos (según evidencia)

- No forzar join `Company` `EMPRESA` por nombre: el profiling muestra que no hay coincidencias confiables.
- Preparar staging normalizado para BD/joins posteriores en `02_data_cleaning/01_raw_normalizado_export.ipynb` (export a CSV).
- Definir estrategia alternativa de integración (p.ej. catálogo unificado de empresas / matching asistido / llaves adicionales).

1.13 9. Universo unificado de empresas (LEADS HORAS) y verificación contra SCVS

> Objetivo: como no hay coincidencias exactas entre ambas fuentes, construimos el **universo unificado** de empresas (a partir de ambas) y evaluamos cobertura en una fuente externa (SCVS / Superintendencia de Compañías).

- Esta sección **no exporta** CSV; solo genera dataframes y métricas para el análisis.

[44]:
```
# === 9.a Limpieza LEADS (Company) ===
```

```python
# Resultado esperado: DataFrame `leads_companies_clean` con Company raw + normalizada (distinct por normalizada)

from pathlib import Path
import re
import unicodedata
import numpy as np
import pandas as pd


def _strip_accents(text: str) -> str:
    return "".join(
        ch for ch in unicodedata.normalize("NFKD", text)
        if not unicodedata.combining(ch)
    )


def normalize_company_name(value) -> str:
    """Normaliza nombres de empresa para comparación (no para mostrar)."""
    if value is None:
        return ""
    if isinstance(value, float) and np.isnan(value):
        return ""

    s = str(value).strip()
    if not s:
        return ""

    s = _strip_accents(s)
    s = s.upper()
    s = re.sub(r"[^A-Z0-9 ]+", " ", s)

    # Quitar sufijos/palabras frecuentes que distorsionan el matching
    s = re.sub(
        r"\b(SA|S\s*A|S\.A\.?|S\.A\.S\.?|SAS|LTDA|CIA|C\.?IA\.?|COMPANIA|COMPAÑIA|CORP|INC|LLC|C\.L\.?|C\.?LTDA\.?|\&|Y)\b",
        " ",
        s,
    )
    s = re.sub(r"\b(DE|DEL|LA|EL|LOS|LAS)\b", " ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s


if "df_leads" not in globals():
    raise NameError("No existe `df_leads` en memoria. Ejecuta la sección de carga de datasets (Sección 4).")
```

```python
if "Company" not in df_leads.columns:
    raise KeyError("La columna `Company` no existe en `df_leads`.")

_company_raw = (
    df_leads["Company"]
    .astype("string")
    .fillna("")
    .map(lambda x: x.strip())
    .replace("", pd.NA)
    .dropna()
    )

leads_companies_clean = (
    pd.DataFrame({"Company_raw": _company_raw})
    .assign(Company_norm=lambda d: d["Company_raw"].map(normalize_company_name))
    .query("Company_norm != ''")
    .drop_duplicates(subset=["Company_norm"], keep="first")
    .sort_values("Company_norm")
    .reset_index(drop=True)
    )

print("[LEADS] Filas originales (no vacías):", int(_company_raw.shape[0]))
print("[LEADS] Empresas distinct (por Company_norm):",␣
 ↪int(leads_companies_clean.shape[0]))

display(leads_companies_clean.head(20))

# --- Export CSV (para descargar desde VS Code) ---
# Guardar en outputs/ relativo al cwd (si estás dentro de␣
 ↪01_data_ingestion_enrichment),
# o en 01_data_ingestion_enrichment/outputs si el cwd es el root del repo.
OUT_DIR = Path("outputs") if Path.cwd().name == "01_data_ingestion_enrichment"␣
 ↪else (Path("01_data_ingestion_enrichment") / "outputs")
OUT_DIR.mkdir(parents=True, exist_ok=True)

out_path = OUT_DIR / "leads_companies_clean.csv"
leads_companies_clean.to_csv(out_path, index=False, encoding="utf-8-sig")
print(f"[LEADS] CSV guardado en: {out_path.resolve()}")
```

```
[LEADS] Filas originales (no vacías): 440
[LEADS] Empresas distinct (por Company_norm): 326
```

	Company_raw	Company_norm
0	7-ELEVEN MEXICO	7 ELEVEN MEXICO
1	ABBOTT	ABBOTT
2	Abbvie	ABBVIE
3	ACCO BRANDS	ACCO BRANDS

```
4   ACH FOOD COMPANIES, INC     ACH FOOD COMPANIES
5                  ACTINVER               ACTINVER
6                ADAMANTINE             ADAMANTINE
7              AFP Genesis            AFP GENESIS
8                       AIG                    AIG
9                     Akros                  AKROS
10              ALCIONE MX             ALCIONE MX
11    AlimentosAlConsumidor  ALIMENTOSALCONSUMIDOR
12           ALLIANZ MÉXICO         ALLIANZ MEXICO
13       ALPEZZI CHOCOLATE      ALPEZZI CHOCOLATE
14                  Alpina                 ALPINA
15       Alvarez Barba S.A          ALVAREZ BARBA
16                AMBROSIA               AMBROSIA
17    ANHEUSER -BUSCH INBEV   ANHEUSER BUSCH INBEV
18                La Anita                 ANITA
19                  APOTEX                 APOTEX
```

```
[LEADS] CSV guardado en: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_dat
a_ingestion_enrichment\outputs\leads_companies_clean.csv
```

[45]:
```python
# === 9.b Limpieza HORAS (EMPRESA) ===
# Resultado esperado: DataFrame `horas_empresas_clean` con EMPRESA raw +␣
 ↪normalizada (distinct por normalizada)

from pathlib import Path
import pandas as pd

if "df_horas" not in globals():
    raise NameError("No existe `df_horas` en memoria. Ejecuta la sección de␣
 ↪carga de datasets (Sección 4).")

if "EMPRESA" not in df_horas.columns:
    raise KeyError("La columna `EMPRESA` no existe en `df_horas`.")

_empresa_raw = (
    df_horas["EMPRESA"]
    .astype("string")
    .fillna("")
    .map(lambda x: x.strip())
    .replace("", pd.NA)
    .dropna()
    )

horas_empresas_clean = (
    pd.DataFrame({"EMPRESA_raw": _empresa_raw})
    .assign(EMPRESA_norm=lambda d: d["EMPRESA_raw"].map(normalize_company_name))
    .query("EMPRESA_norm != ''")
    .drop_duplicates(subset=["EMPRESA_norm"], keep="first")
```

```
    .sort_values("EMPRESA_norm")
    .reset_index(drop=True)
    )

print("[HORAS] Filas originales (no vacías):", int(_empresa_raw.shape[0]))
print("[HORAS] Empresas distinct (por EMPRESA_norm):", int(horas_empresas_clean.
 ↪shape[0]))

display(horas_empresas_clean.head(20))

# --- Export CSV (para descargar desde VS Code) ---
OUT_DIR = Path("outputs") if Path.cwd().name == "01_data_ingestion_enrichment"␣
 ↪else (Path("01_data_ingestion_enrichment") / "outputs")
OUT_DIR.mkdir(parents=True, exist_ok=True)

out_path = OUT_DIR / "horas_empresas_clean.csv"
horas_empresas_clean.to_csv(out_path, index=False, encoding="utf-8-sig")
print(f"[HORAS] CSV guardado en: {out_path.resolve()}")

# --- CSV compilado de empresas (LEADS  HORAS) ---
if "leads_companies_clean" in globals() and leads_companies_clean is not None␣
 ↪and not leads_companies_clean.empty:
    leads_u = (
        leads_companies_clean
        .rename(columns={"Company_raw": "name_raw", "Company_norm":␣
 ↪"name_norm"})
        .assign(source="LEADS")
        [["source", "name_raw", "name_norm"]]
    )
    horas_u = (
        horas_empresas_clean
        .rename(columns={"EMPRESA_raw": "name_raw", "EMPRESA_norm":␣
 ↪"name_norm"})
        .assign(source="HORAS")
        [["source", "name_raw", "name_norm"]]
    )

    universe_all = pd.concat([leads_u, horas_u], ignore_index=True)
    universe_distinct = (
        universe_all
        .groupby("name_norm", as_index=False)
        .agg(
            name_raw=("name_raw", "first"),
            sources=("source", lambda s: ",".join(sorted(set(map(str, s))))),
            n_rows=("source", "size"),
        )
        .sort_values("name_norm")
```

```
        .reset_index(drop=True)
    )

    out_universe = OUT_DIR / "empresas_universe_compilado.csv"
    universe_distinct.to_csv(out_universe, index=False, encoding="utf-8-sig")
    print(f"[UNIVERSO] CSV compilado guardado en: {out_universe.resolve()}")
    display(universe_distinct.head(20))
else:
    print("[UNIVERSO] Nota: no existe `leads_companies_clean` (ejecuta primero␣
 ↪9.a) para generar el compilado LEADS  HORAS.")
```

```
[HORAS] Filas originales (no vacías): 412
[HORAS] Empresas distinct (por EMPRESA_norm): 119
```

	EMPRESA_raw	EMPRESA_norm
0	3dpharma	3DPHARMA
1	Adium	ADIUM
2	Almexa	ALMEXA
3	Alper Seguros	ALPER SEGUROS
4	Arauco	ARAUCO
5	Aseguradora del Sur	ASEGURADORA SUR
6	Asesoría y Control	ASESORIA CONTROL
7	AutoShare	AUTOSHARE
8	AVIS	AVIS
9	BAC	BAC
10	Banco del Austro	BANCO AUSTRO
11	Banco Financiero	BANCO FINANCIERO
12	BANRED	BANRED
13	Bekaert	BEKAERT
14	Bemis	BEMIS
15	Bitso	BITSO
16	Bloomin Brands	BLOOMIN BRANDS
17	BMI	BMI
18	Boston BSCI	BOSTON BSCI
19	CEPSA	CEPSA

```
[HORAS] CSV guardado en: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_dat
a_ingestion_enrichment\outputs\horas_empresas_clean.csv
[UNIVERSO] CSV compilado guardado en: E:\TESIS MAESTRIA\Desarrollo_clustering_ma
estria\01_data_ingestion_enrichment\outputs\empresas_universe_compilado.csv
```

	name_norm	name_raw	sources	n_rows
0	3DPHARMA	3dpharma	HORAS	1
1	7 ELEVEN MEXICO	7-ELEVEN MEXICO	LEADS	1
2	ABBOTT	ABBOTT	LEADS	1
3	ABBVIE	Abbvie	LEADS	1
4	ACCO BRANDS	ACCO BRANDS	LEADS	1
5	ACH FOOD COMPANIES	ACH FOOD COMPANIES, INC	LEADS	1
6	ACTINVER	ACTINVER	LEADS	1

7	ADAMANTINE	ADAMANTINE	LEADS	1
8	ADIUM	Adium	HORAS	1
9	AFP GENESIS	AFP Genesis	LEADS	1
10	AIG	AIG	LEADS	1
11	AKROS	Akros	LEADS	1
12	ALCIONE MX	ALCIONE MX	LEADS	1
13	ALIMENTOSALCONSUMIDOR	AlimentosAlConsumidor	LEADS	1
14	ALLIANZ MEXICO	ALLIANZ MÉXICO	LEADS	1
15	ALMEXA	Almexa	HORAS	1
16	ALPER SEGUROS	Alper Seguros	HORAS	1
17	ALPEZZI CHOCOLATE	ALPEZZI CHOCOLATE	LEADS	1
18	ALPINA	Alpina	LEADS	1
19	ALVAREZ BARBA	Alvarez Barba S.A	LEADS	1

[29]:
```
# === 9.c Verificación RUC para LEADS vs Super (SCVS) ===
# Salida principal: `leads_ruc_exact` (match exacto) y `leads_ruc_sugerido`␣
 ↪(match por similitud para no-matcheados)

from pathlib import Path
import os
import pandas as pd
import numpy as np
import re

# Carpeta con archivos SCVS/Super. La buscamos en ubicaciones típicas.
_CANDIDATE_SUPER_DIRS = [
    Path("data_super_compañias"),
    Path("01_data_ingestion_enrichment") / "data_super_compañias",
]
SUPER_DIR = next((p for p in _CANDIDATE_SUPER_DIRS if p.exists()),␣
 ↪_CANDIDATE_SUPER_DIRS[0])


def _normalize_ruc(value) -> str:
    """Normaliza RUC a solo dígitos. Ecuador típico: 13 dígitos."""
    if value is None:
        return ""
    if isinstance(value, float) and np.isnan(value):
        return ""
    s = str(value).strip()
    if not s:
        return ""
    s = s.replace(".0", "")
    s = re.sub(r"\D+", "", s)
    return s
```

```python
def _pick_col(cols, patterns):
    cols_u = [str(c).upper() for c in cols]
    for pat in patterns:
        for i, cu in enumerate(cols_u):
            if pat in cu:
                return cols[i]
    return None


def _detect_excel_header_row(fp: Path, max_rows: int = 80) -> int:
    """Detecta la fila de encabezado en Excels tipo reporte (títulos arriba)."""
    try:
        raw = pd.read_excel(fp, header=None, nrows=max_rows)
    except Exception:
        return 0

    best_i = 0
    best_score = -1

    must_have_any = ["RUC", "IDENTIFIC"]
    name_tokens = ["NOMBRE", "RAZON", "RAZÓN", "DENOM", "COMPAÑ"]

    for i in range(len(raw)):
        vals = [v for v in raw.iloc[i].tolist() if str(v).lower() != "nan"]
        if not vals:
            continue
        row_text = " ".join(map(str, vals)).upper()

        score = 0
        score += sum(tok in row_text for tok in must_have_any) * 5
        score += sum(tok in row_text for tok in name_tokens) * 3
        score += ("EXPEDIENTE" in row_text) * 2
        score += ("ESTADO" in row_text) * 1
        score += ("TIPO" in row_text) * 1

        if score > best_score:
            best_score = score
            best_i = i

    return int(best_i)


def _load_super_companies(folder: Path):
    """Carga múltiples archivos de Super/SCVS y arma un catálogo (nombre_norm␣
 ↪-> ruc).

    - Soporta Excels tipo reporte (detecta fila de encabezado).
```

```
    - Filtra a RUCs plausibles (13 dígitos) para evitar falsos positivos.
    """
    if not folder.exists():
        raise FileNotFoundError(f"No existe la carpeta: {folder}")

    files = sorted([p for p in folder.iterdir() if p.suffix.lower() in {".
↪xlsx", ".xls", ".csv"}])
    if not files:
        raise FileNotFoundError(f"No se encontraron archivos .xlsx/.xls/.csv en␣
↪{folder}")

    frames = []
    for fp in files:
        try:
            if fp.suffix.lower() == ".csv":
                df = pd.read_csv(fp)
            else:
                header_row = _detect_excel_header_row(fp)
                df = pd.read_excel(fp, header=header_row)
        except Exception as e:
            print(f"[SUPER] No se pudo leer {fp.name}: {e}")
            continue

        if df is None or df.empty:
            continue

        # Limpiar nombres de columnas
        df.columns = [str(c).strip() for c in df.columns]

        name_col = _pick_col(
            df.columns,
            patterns=[
                "RAZON", "RAZÓN", "RAZON SOCIAL", "RAZONSOCIAL",
                "NOMBRE", "DENOM", "DENOMIN", "COMPAÑ", "COMPAN",
                "EMPRESA", "INSTITUC", "ORGANIZ",
            ],
        )
        ruc_col = _pick_col(df.columns, patterns=["RUC", "IDENTIFIC",␣
↪"IDENTIFICACIÓN", "IDENTIFICACION", "CEDULA", "CÉDULA", "CI"])

        if name_col is None:
            obj_cols = [c for c in df.columns if str(df[c].dtype) in ("object",␣
↪"string")]
            name_col = obj_cols[0] if obj_cols else None

        if ruc_col is None:
```

```
            # fallback: escoger columna con más valores que parecen RUC (13 dígitos)
            best = None
            best_score = -1.0
            for c in df.columns:
                s = df[c].map(_normalize_ruc)
                if not hasattr(s, "str"):
                    continue
                non_empty = float((s != "").mean())
                len13 = float((s.str.len() == 13).mean())
                score = non_empty + 3.0 * len13
                if score > best_score:
                    best_score = score
                    best = c
            ruc_col = best

        if name_col is None or ruc_col is None:
            print(f"[SUPER] Saltando {fp.name}: no pude detectar columnas nombre/RUC")
            continue

        tmp = pd.DataFrame({
            "super_source": fp.name,
            "super_name_raw": df[name_col].astype("string"),
            "super_ruc_raw": df[ruc_col],
        })
        tmp["super_name_raw"] = tmp["super_name_raw"].fillna("").map(lambda x: x.strip())
        tmp["super_ruc"] = tmp["super_ruc_raw"].map(_normalize_ruc)

        # Mantener solo RUCs plausibles (Ecuador = 13 dígitos)
        tmp = tmp[tmp["super_ruc"].astype("string").str.len() == 13].copy()

        tmp["super_name_norm"] = tmp["super_name_raw"].map(normalize_company_name)

        # Mantener solo nombres razonables (con letras)
        tmp = tmp[tmp["super_name_norm"].str.contains(r"[A-Z]", regex=True, na=False)].copy()
        tmp = tmp[tmp["super_name_norm"].str.len() >= 3].copy()

        if tmp.empty:
            continue

        frames.append(tmp[["super_source", "super_name_raw", "super_name_norm", "super_ruc"]])
```

```
    if not frames:
        raise ValueError("Se leyeron archivos, pero no se pudo armar un␣
 ↪catálogo válido (encabezado/RUC/nombres).")

    super_all = pd.concat(frames, ignore_index=True)

    # Si hay duplicados por nombre_norm con distinto ruc, nos quedamos con el␣
 ↪más frecuente
    super_lookup = (
        super_all
        .groupby(["super_name_norm", "super_ruc"], as_index=False)
        .size()
        .sort_values(["super_name_norm", "size"], ascending=[True, False])
    )
    super_best = super_lookup.drop_duplicates(subset=["super_name_norm"],␣
 ↪keep="first")[["super_name_norm", "super_ruc"]]

    return super_all, super_best


if "leads_companies_clean" not in globals():
    raise NameError("No existe `leads_companies_clean`. Ejecuta primero la␣
 ↪celda 9.a (limpieza LEADS).")

super_all, super_best = _load_super_companies(SUPER_DIR)
print("[SUPER] Carpeta usada:", str(SUPER_DIR))
print("[SUPER] Registros (con nombre y RUC 13d):", int(super_all.shape[0]))
print("[SUPER] Nombres distinct (lookup):", int(super_best.shape[0]))

# Match exacto por nombre normalizado
leads_ruc_exact = (
    leads_companies_clean
    .merge(super_best, left_on="Company_norm", right_on="super_name_norm",␣
 ↪how="left")
    .drop(columns=["super_name_norm"])
    .rename(columns={"super_ruc": "RUC"})
)

n_total = int(leads_ruc_exact.shape[0])
n_con_ruc = int(leads_ruc_exact["RUC"].notna().sum())
print(f"[LEADS] Total empresas distinct: {n_total}")
print(f"[LEADS] Con RUC (match exacto): {n_con_ruc} ({(n_con_ruc/
 ↪max(n_total,1))*100:.1f}%)")
```

```
# Para aumentar cobertura: sugerencia fuzzy solo para las que NO tienen RUC por␣
 ↪match exacto
# Por defecto bajamos a 80 para explorar cobertura (ajustable con␣
 ↪SCVS_FUZZY_THRESHOLD)
UMBRAL_SIMILITUD = int(os.getenv("SCVS_FUZZY_THRESHOLD", "80"))
unmatched = leads_ruc_exact[leads_ruc_exact["RUC"].isna()].copy()

choices = super_best.copy()
choices["name_norm"] = choices["super_name_norm"]
choices = choices[["name_norm", "super_ruc"]].dropna().drop_duplicates().
 ↪reset_index(drop=True)
choice_names = choices["name_norm"].tolist()

rows = []
try:
    from rapidfuzz import process, fuzz
    for _, r in unmatched.iterrows():
        q = r["Company_norm"]
        if not q:
            continue
        best = process.extractOne(q, choice_names, scorer=fuzz.token_set_ratio)
        if best is None:
            continue
        best_name, score, _ = best
        if float(score) >= UMBRAL_SIMILITUD:
            ruc = choices.loc[choices["name_norm"] == best_name, "super_ruc"].
 ↪iloc[0]
            rows.append({
                "Company_raw": r["Company_raw"],
                "Company_norm": q,
                "best_super_name_norm": best_name,
                "score": float(score),
                "RUC": ruc,
            })
    engine = "rapidfuzz"
except Exception:
    from difflib import SequenceMatcher

    def _ratio(a, b):
        return SequenceMatcher(None, a, b).ratio() * 100

    for _, r in unmatched.iterrows():
        q = r["Company_norm"]
        if not q:
            continue
        best_name = None
        best_score = -1.0
```

```
        for cand in choice_names:
            sc = _ratio(q, cand)
            if sc > best_score:
                best_score = sc
                best_name = cand
        if best_name is not None and best_score >= UMBRAL_SIMILITUD:
            ruc = choices.loc[choices["name_norm"] == best_name, "super_ruc"].
↪iloc[0]
            rows.append({
                "Company_raw": r["Company_raw"],
                "Company_norm": q,
                "best_super_name_norm": best_name,
                "score": float(best_score),
                "RUC": ruc,
            })
    engine = "difflib"

leads_ruc_sugerido = pd.DataFrame(rows)
if leads_ruc_sugerido.empty:
    leads_ruc_sugerido = pd.DataFrame(columns=["Company_raw", "Company_norm",␣
↪"best_super_name_norm", "score", "RUC"])
else:
    leads_ruc_sugerido = leads_ruc_sugerido.sort_values("score",␣
↪ascending=False).reset_index(drop=True)

print(f"[LEADS] Motor fuzzy: {engine}")
print(f"[LEADS] Sugerencias fuzzy con score >= {UMBRAL_SIMILITUD}:␣
↪{int(leads_ruc_sugerido.shape[0])}")

display(leads_ruc_exact.head(20))
print("\n--- Sugerencias fuzzy (top 20) ---")
display(leads_ruc_sugerido.head(20))
```

```
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
```

```
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
  warn("Workbook contains no default style, apply openpyxl's default")

[SUPER] Carpeta usada: data_super_compañias
[SUPER] Registros (con nombre y RUC 13d): 215291
[SUPER] Nombres distinct (lookup): 214446
[LEADS] Total empresas distinct: 326
[LEADS] Con RUC (match exacto): 6 (1.8%)
[LEADS] Motor fuzzy: rapidfuzz
[LEADS] Sugerencias fuzzy con score >= 80: 181

               Company_raw          Company_norm           RUC
0          7-ELEVEN MEXICO       7 ELEVEN MEXICO           NaN
1                   ABBOTT                ABBOTT           NaN
2                   Abbvie                ABBVIE           NaN
3              ACCO BRANDS           ACCO BRANDS           NaN
4   ACH FOOD COMPANIES, INC    ACH FOOD COMPANIES           NaN
```

```
5                  ACTINVER              ACTINVER            NaN
6                ADAMANTINE            ADAMANTINE            NaN
7              AFP Genesis           AFP GENESIS            NaN
8                      AIG                   AIG            NaN
9                    Akros                 AKROS  1791148800001
10               ALCIONE MX            ALCIONE MX            NaN
11   AlimentosAlConsumidor  ALIMENTOSALCONSUMIDOR            NaN
12          ALLIANZ MÉXICO        ALLIANZ MEXICO            NaN
13       ALPEZZI CHOCOLATE     ALPEZZI CHOCOLATE            NaN
14                  Alpina                ALPINA            NaN
15       Alvarez Barba S.A          ALVAREZ BARBA  1790360741001
16                 AMBROSIA              AMBROSIA            NaN
17   ANHEUSER -BUSCH INBEV   ANHEUSER BUSCH INBEV            NaN
18                La Anita                 ANITA            NaN
19                  APOTEX                APOTEX            NaN


--- Sugerencias fuzzy (top 20) ---

                     Company_raw                Company_norm                 ␣
 ↪         best_super_name_norm  score             RUC
0                          ABBOTT                      ABBOTT                 ␣
 ↪  ABBOTT LABORATORIOS ECUADOR  100.0  0990000670001
1                            AIG                         AIG            AIG␣
 ↪METROPOLITANA SEGUROS REASEGUROS  100.0  1790475247001
2                    AFP Genesis                 AFP GENESIS  AFP GENESIS␣
 ↪ADMINISTRADORA FONDOS FIDEICOMISOS  100.0  0991307605001
3                          Alpina                      ALPINA                 ␣
 ↪    ALPINA BEVERAGE ALPINAGUA  100.0  0991371605001
4                        AMBROSIA                    AMBROSIA                 ␣
 ↪    AMBROSIA BIO WINE S B I C  100.0  1191797340001
5         BACCHUS CONSULTING GROUP    BACCHUS CONSULTING GROUP                 ␣
 ↪                   CONSULTING  100.0  0993372635001
6                           BALL                        BALL                 ␣
 ↪                    BALL 9 S  100.0  0993383152001
7                       La Anita                       ANITA   COMERCIAL␣
 ↪IMPORTADORA SANTA ANITA C A IMSANIT  100.0  0990085188001
8                          BAXTER                      BAXTER                 ␣
 ↪               BAXTER ECUADOR  100.0  1791253531001
9                          AXIONLOG                    AXIONLOG                 ␣
 ↪             AXIONLOG ECUADOR  100.0  0992991178001
10  Asiauto S.A Kia Motors Ecuador  ASIAUTO KIA MOTORS ECUADOR                 ␣
 ↪                       ASIAUTO  100.0  1791754115001
11                       Confiteca                   CONFITECA                 ␣
 ↪                CONFITECA C A  100.0  1790084604001
12                         Directv                     DIRECTV                 ␣
 ↪            DIRECTV ECUADOR C  100.0  1792067782001
```

```
13                            Chubb                          CHUBB
 ↪        CHUBB SEGUROS ECUADOR  100.0  1790516008001
14                            Claro                          CLARO
 ↪           MINERA RIO CLARO C  100.0  0791844366001
15                              CFB                            CFB
 ↪CENTRO FERRETERO BAMBOO CFB S  100.0  1793230370001
16                              CBC                            CBC
 ↪  CENTRAL BARBER COMPAN CBC S  100.0  0993388951001
17                           Carval                         CARVAL
 ↪    CARVAL ECUADOR CARVALECSA  100.0  0993107859001
18                          CARGILL                        CARGILL
 ↪CARGILL AQUANUTRITION ECUADOR  100.0  0993376610001
19              Biopas Laboratorios           BIOPAS LABORATORIOS
 ↪          LABORATORIOS BIOPAS  100.0  1791891112001
```

[46]:
```python
# === 9.c.1 Verificación RUC en Padrón SRI para LEADS huérfanas ===
# Objetivo: segundo intento de match (fuzzy) para LEADS sin RUC (huérfanas) contra el padrón SRI.
# Nota de performance/RAM: el padrón puede ser masivo. Este bloque lee por chunks y construye un catálogo deduplicado por nombre normalizado.

from pathlib import Path
import os
import pandas as pd
import numpy as np
import re

# --- Requisitos de entrada ---
if "leads_ruc_exact" not in globals():
    raise NameError("No existe `leads_ruc_exact`. Ejecuta primero 9.c.")
if "normalize_company_name" not in globals():
    raise NameError("No existe `normalize_company_name` en memoria.")

# Huérfanas: LEADS sin RUC tras SCVS
unmatched_leads = leads_ruc_exact[leads_ruc_exact["RUC"].isna()].copy()
print("[LEADS/SRI] Huérfanas (sin RUC por SCVS exacto):", int(unmatched_leads.shape[0]))

# --- Configuración ---
UMBRAL_SRI = int(os.getenv("UMBRAL_SRI", "80"))
SRI_CHUNK_ROWS = int(os.getenv("SRI_CHUNK_ROWS", "200000"))

# Ruta: carpeta con múltiples CSV (SRI por provincias u otros)
SRI_DIR = Path(r"E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\data_SRI")
if not SRI_DIR.exists():
    # fallback relativo (si el repo se movió o se ejecuta desde otra carpeta)
```

```
    SRI_DIR = Path("01_data_ingestion_enrichment") / "data_SRI"
if not SRI_DIR.exists():
    raise FileNotFoundError(f"No existe SRI_DIR: {SRI_DIR}")

sri_files = sorted([p for p in SRI_DIR.iterdir() if p.suffix.lower() == ".csv"])
if not sri_files:
    raise FileNotFoundError(f"No hay archivos .csv en: {SRI_DIR}")
print("[SRI] Archivos CSV detectados:", len(sri_files))

# --- Utilitarios ---
def _normalize_ruc(value) -> str:
    if value is None:
        return ""
    if isinstance(value, float) and np.isnan(value):
        return ""
    s = str(value).strip()
    if not s:
        return ""
    s = s.replace(".0", "")
    s = re.sub(r"\D+", "", s)
    return s

def _guess_sep(fp: Path) -> str:
    # Heurística simple para detectar delimitador (',' vs ';') sin cargar el␣
 ↪archivo completo
    try:
        with fp.open("rb") as f:
            head = f.read(4096).decode("utf-8", errors="ignore")
    except Exception:
        return ","
    return ";" if head.count(";") > head.count(",") else ","

def _pick_col(cols, patterns):
    cols_u = [str(c).upper() for c in cols]
    for pat in patterns:
        for i, cu in enumerate(cols_u):
            if pat in cu:
                return cols[i]
    return None

def _read_csv_chunks(fp: Path, usecols: list[str], sep: str, encoding: str):
    # Compatibilidad pandas: on_bad_lines (nuevo) vs error_bad_lines (viejo)
    try:
        return pd.read_csv(
            fp,
            usecols=usecols,
            dtype=str,
```

```
            sep=sep,
            chunksize=SRI_CHUNK_ROWS,
            low_memory=True,
            encoding=encoding,
            on_bad_lines="skip",
        )
    except TypeError:
        return pd.read_csv(
            fp,
            usecols=usecols,
            dtype=str,
            sep=sep,
            chunksize=SRI_CHUNK_ROWS,
            low_memory=True,
            encoding=encoding,
            error_bad_lines=False,
            warn_bad_lines=True,
        )

def _read_sri_in_chunks(fp: Path):
    """Devuelve un iterador de chunks con columnas de RUC y razón social␣
 ↪(nombres variables)."""
    sep = _guess_sep(fp)
    encodings = ["utf-8", "latin1"]

    # Intento 1: nombres esperados (pueden variar)
    preferred = ["NUMERO_RUC", "RAZON_SOCIAL"]
    for enc in encodings:
        try:
            return _read_csv_chunks(fp, usecols=preferred, sep=sep,␣
 ↪encoding=enc)
        except Exception:
            pass

    # Intento 2: detectar equivalentes leyendo solo encabezados
    try:
        cols = pd.read_csv(fp, nrows=0, sep=sep, encoding="latin1").columns.
 ↪tolist()
    except Exception as e:
        raise RuntimeError(f"No pude leer encabezado de {fp.name}: {e}") from e

    ruc_col = _pick_col(cols, patterns=["NUMERO_RUC", "RUC", "IDENTIFIC",␣
 ↪"IDENTIFICACION", "IDENTIFICACIÓN"])
    name_col = _pick_col(cols, patterns=["RAZON_SOCIAL", "RAZON", "RAZÓN",␣
 ↪"NOMBRE"])
    if ruc_col is None or name_col is None:
```

```
        raise RuntimeError(f"No pude detectar columnas RUC/RAZON en {fp.name}. Columnas: {cols[:20]}")

    for enc in encodings:
        try:
            return _read_csv_chunks(fp, usecols=[ruc_col, name_col], sep=sep, encoding=enc)
        except Exception:
            pass

    raise RuntimeError(f"No pude leer {fp.name} con columnas detectadas.")

# --- Construir catálogo maestro sri_best (deduplicado por nombre normalizado) ---
count_map: dict[tuple[str, str], int] = {}  # (name_norm, ruc) -> frecuencia acumulada
n_rows_seen = 0
n_rows_kept = 0

for i, fp in enumerate(sri_files, start=1):
    print(f"[SRI] ({i}/{len(sri_files)}) Leyendo: {fp.name}")
    try:
        chunk_iter = _read_sri_in_chunks(fp)
    except Exception as e:
        print(f"[SRI]   Saltado (no legible): {fp.name} -> {e}")
        continue

    for chunk in chunk_iter:
        if chunk is None or chunk.empty:
            continue
        n_rows_seen += int(len(chunk))

        cols = list(chunk.columns)
        ruc_col = _pick_col(cols, patterns=["NUMERO_RUC", "RUC", "IDENTIFIC"]) or cols[0]
        name_col = _pick_col(cols, patterns=["RAZON_SOCIAL", "RAZON", "RAZÓN", "NOMBRE"])
        if name_col is None:
            name_col = cols[1] if len(cols) > 1 else cols[0]

        tmp = pd.DataFrame({
            "ruc_raw": chunk[ruc_col],
            "razon_raw": chunk[name_col],
        })
        tmp["ruc"] = tmp["ruc_raw"].map(_normalize_ruc)
```

```
        tmp["sri_name_raw"] = tmp["razon_raw"].astype("string").fillna("").
 ↪map(lambda x: x.strip())
        tmp["sri_name_norm"] = tmp["sri_name_raw"].map(normalize_company_name)

        # Filtros mínimos para calidad/velocidad
        tmp = tmp[tmp["ruc"].astype("string").str.len() == 13].copy()
        tmp = tmp[tmp["sri_name_norm"].str.contains(r"[A-Z]", regex=True,␣
 ↪na=False)].copy()
        tmp = tmp[tmp["sri_name_norm"].str.len() >= 3].copy()
        if tmp.empty:
            continue

        n_rows_kept += int(len(tmp))
        grp = tmp.groupby(["sri_name_norm", "ruc"]).size()
        for (name_norm, ruc), cnt in grp.items():
            key = (str(name_norm), str(ruc))
            count_map[key] = count_map.get(key, 0) + int(cnt)

print("[SRI] Filas leídas (aprox):", n_rows_seen)
print("[SRI] Filas retenidas tras filtros:", n_rows_kept)
print("[SRI] Pares (nombre_norm, ruc) únicos:", len(count_map))

if not count_map:
    raise ValueError("No se pudo construir catálogo SRI (count_map vacío).␣
 ↪Revisa separador/encoding/columnas.")

sri_lookup = (
    pd.DataFrame([
        {"sri_name_norm": k[0], "RUC": k[1], "freq": v}
        for k, v in count_map.items()
    ])
    .sort_values(["sri_name_norm", "freq"], ascending=[True, False])
    .reset_index(drop=True)
)

# Para cada nombre_norm, nos quedamos con el RUC más frecuente (similar a␣
 ↪super_best)
sri_best = (
    sri_lookup
    .drop_duplicates(subset=["sri_name_norm"], keep="first")
    [["sri_name_norm", "RUC"]]
    .reset_index(drop=True)
)
print("[SRI] sri_best (nombres distinct):", int(sri_best.shape[0]))

# --- Fuzzy matching: LEADS huérfanas -> SRI ---
try:
```

```
    from rapidfuzz import process, fuzz
except Exception as e:
    raise ImportError("Falta rapidfuzz. Instala con: pip install rapidfuzz")␣
 ↪from e

choice_names = sri_best["sri_name_norm"].astype("string").dropna().tolist()
rows = []
for _, r in unmatched_leads.iterrows():
    q = r.get("Company_norm", "")
    if q is None:
        continue
    q = str(q).strip()
    if not q:
        continue
    best = process.extractOne(q, choice_names, scorer=fuzz.token_set_ratio)
    if best is None:
        continue
    best_name, score, _ = best
    if float(score) >= float(UMBRAL_SRI):
        ruc = sri_best.loc[sri_best["sri_name_norm"] == best_name, "RUC"].
 ↪iloc[0]
        rows.append({
            "Company_raw": r.get("Company_raw", ""),
            "Company_norm": q,
            "best_sri_name_norm": best_name,
            "score": float(score),
            "RUC": str(ruc),
        })

leads_sri_sugerido = pd.DataFrame(rows)
if leads_sri_sugerido.empty:
    leads_sri_sugerido = pd.DataFrame(columns=["Company_raw", "Company_norm",␣
 ↪"best_sri_name_norm", "score", "RUC"])
else:
    leads_sri_sugerido = leads_sri_sugerido.sort_values("score",␣
 ↪ascending=False).reset_index(drop=True)

print(f"[LEADS/SRI] Umbral fuzzy SRI (UMBRAL_SRI): {UMBRAL_SRI}")
print(f"[LEADS/SRI] Sugerencias fuzzy SRI: {int(leads_sri_sugerido.shape[0])}")
display(leads_sri_sugerido.head(20))
```

```
[LEADS/SRI] Huérfanas (sin RUC por SCVS exacto): 320
[SRI] Archivos CSV detectados: 26
[SRI] (1/26) Leyendo: SRI_Catastro_Empresas_Fantasmas.csv
[SRI] (2/26) Leyendo: SRI_MERCADOSENLINEA.csv
[SRI] (3/26) Leyendo: SRI_RUC_Azuay.csv
[SRI] (4/26) Leyendo: SRI_RUC_Bolivar.csv
```

```
[SRI] (5/26) Leyendo: SRI_RUC_Carchi.csv
[SRI] (6/26) Leyendo: SRI_RUC_Cañar.csv
[SRI] (7/26) Leyendo: SRI_RUC_Chimborazo.csv
[SRI] (8/26) Leyendo: SRI_RUC_Cotopaxi.csv
[SRI] (9/26) Leyendo: SRI_RUC_El_Oro.csv
[SRI] (10/26) Leyendo: SRI_RUC_Esmeraldas.csv
[SRI] (11/26) Leyendo: SRI_RUC_Galapagos.csv
[SRI] (12/26) Leyendo: SRI_RUC_Guayas.csv
[SRI] (13/26) Leyendo: SRI_RUC_Imbabura.csv
[SRI] (14/26) Leyendo: SRI_RUC_Loja.csv
[SRI] (15/26) Leyendo: SRI_RUC_Los_Rios.csv
[SRI] (16/26) Leyendo: SRI_RUC_Manabi.csv
[SRI] (17/26) Leyendo: SRI_RUC_Morona_Santiago.csv
[SRI] (18/26) Leyendo: SRI_RUC_Napo.csv
[SRI] (19/26) Leyendo: SRI_RUC_Orellana.csv
[SRI] (20/26) Leyendo: SRI_RUC_Pastaza.csv
[SRI] (21/26) Leyendo: SRI_RUC_Pichincha.csv
[SRI] (22/26) Leyendo: SRI_RUC_Santa_Elena.csv
[SRI] (23/26) Leyendo: SRI_RUC_Santo_Domingo.csv
[SRI] (24/26) Leyendo: SRI_RUC_Sucumbios.csv
[SRI] (25/26) Leyendo: SRI_RUC_Tungurahua.csv
[SRI] (26/26) Leyendo: SRI_RUC_Zamora_Chinchipe.csv
[SRI] Filas leídas (aprox): 7996416
[SRI] Filas retenidas tras filtros: 13744
[SRI] Pares (nombre_norm, ruc) únicos: 10890
[SRI] sri_best (nombres distinct): 10890
[LEADS/SRI] Umbral fuzzy SRI (UMBRAL_SRI): 80
[LEADS/SRI] Sugerencias fuzzy SRI: 12

                      Company_raw                    Company_norm
↪                  best_sri_name_norm       score            RUC
0                           La Anita                          ANITA
↪0591753045001 ASOCIACION COMERCIANTES SANTA ANITA  100.000000  0591753045001
1                  ARCA CONTINENTAL               ARCA CONTINENTAL
↪1793211593001 SINDICATO TRABAJADORES EMPRESA A…  100.000000  1793211593001
2                               RB                             RB
↪1793215982001 RB COMUNICACION ESTRATEGICA  100.000000  1793215982001
3                       Telefonica                     TELEFONICA
↪0991408479001 EMPRESA INGENIERIA TELEFONICA EL…  100.000000  0991408479001
4       STE-SOLUCIONES TECNOLÓGICAS    STE SOLUCIONES TECNOLOGICAS
↪1792530911001 SOLUCIONES DIGITALES TECNOLOGICAS   92.000000  1792530911001
5        MTWA SOLUCIONES INTEGRALES     MTWA SOLUCIONES INTEGRALES
↪0993000175001 SOLUCIONES INTEGRALES TECNOLOGIA   89.361702  0993000175001
6   PRODUCTOS DE CONSUMO Z SA DE CV         PRODUCTOS CONSUMO Z CV
↪0992287861001 REPRESENTACION DISTRIBUCION PROD…   87.179487  0992287861001
7                 FGX INTERNACIONAL              FGX INTERNACIONAL
↪0190419800001 TRANSPORTE NACIONAL E INTERNACIONAL   86.666667  0190419800001
```

```
8                    CLG TRANSPORTES                    CLG TRANSPORTES
  ↪   0190149528001 TRANSPORTES QUEZADA   84.615385  0190149528001
9     SOLUCIONES INTEGRALES IKNELIA  SOLUCIONES INTEGRALES IKNELIA
  ↪0993000175001 SOLUCIONES INTEGRALES TECNOLOGIA   84.000000  0993000175001
10                PISA FARMACEÚTICA                PISA FARMACEUTICA
  ↪0991250034001 INDUSTRIA FARMACEUTICA   82.758621  0991250034001
11         BACCHUS CONSULTING GROUP        BACCHUS CONSULTING GROUP
  ↪  1792539773001 ACC CONSULTING GROUP   80.000000  1792539773001
```

[ ]:
```
# === 9.c.2 Verificación RUC por NOMBRE_FANTASIA_COMERCIAL para LEADS ===
# Objetivo: tercer intento de match (fuzzy) usando el nombre comercial/fantasía
  ↪del SRI, solo para huérfanas que siguen sin match por Razón Social.

from pathlib import Path
import os
import pandas as pd
import numpy as np
import re

# --- Requisitos de entrada ---
if "unmatched_leads" not in globals():
    raise NameError("No existe `unmatched_leads`. Ejecuta primero 9.c.1.")
if "leads_sri_sugerido" not in globals():
    raise NameError("No existe `leads_sri_sugerido`. Ejecuta primero 9.c.1.")
if "normalize_company_name" not in globals():
    raise NameError("No existe `normalize_company_name` en memoria.")

UMBRAL_SRI = int(os.getenv("UMBRAL_SRI", "80"))
SRI_CHUNK_ROWS = int(os.getenv("SRI_CHUNK_ROWS", "200000"))

# --- Huérfanas restantes (fase 3): excluir las que ya tuvieron sugerencia por
  ↪Razón Social ---
matched_raw = set(
    leads_sri_sugerido["Company_raw"].astype("string").fillna("").map(lambda x:
  ↪x.strip()).tolist()
    if not leads_sri_sugerido.empty and "Company_raw" in leads_sri_sugerido.
  ↪columns
    else []
 )
unmatched_leads_fase3 = unmatched_leads.copy()
if "Company_raw" in unmatched_leads_fase3.columns:
    unmatched_leads_fase3 =
  ↪unmatched_leads_fase3[~unmatched_leads_fase3["Company_raw"].astype("string").
  ↪fillna("").map(lambda x: x.strip()).isin(matched_raw)].copy()
print("[LEADS/FANTASIA] Huérfanas restantes para fase 3:",
  ↪int(unmatched_leads_fase3.shape[0]))
```

```
# --- Ruta SRI (carpeta CSV) ---
SRI_DIR = Path(r"E:\TESIS␣
 ↪MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\data_SRI")
if not SRI_DIR.exists():
    SRI_DIR = Path("01_data_ingestion_enrichment") / "data_SRI"
if not SRI_DIR.exists():
    raise FileNotFoundError(f"No existe SRI_DIR: {SRI_DIR}")

sri_files = sorted([p for p in SRI_DIR.iterdir() if p.suffix.lower() == ".csv"])
if not sri_files:
    raise FileNotFoundError(f"No hay archivos .csv en: {SRI_DIR}")
print("[SRI/FANTASIA] Archivos CSV detectados:", len(sri_files))

# --- Utilitarios (compactos; no asumen delimitador fijo) ---
def _normalize_ruc(value) -> str:
    if value is None:
        return ""
    if isinstance(value, float) and np.isnan(value):
        return ""
    s = str(value).strip()
    if not s:
        return ""
    s = s.replace(".0", "")
    s = re.sub(r"\D+", "", s)
    return s

def _guess_sep(fp: Path) -> str:
    try:
        with fp.open("rb") as f:
            head = f.read(4096).decode("utf-8", errors="ignore")
    except Exception:
        return ","
    return ";" if head.count(";") > head.count(",") else ","

def _pick_col(cols, patterns):
    cols_u = [str(c).upper() for c in cols]
    for pat in patterns:
        for i, cu in enumerate(cols_u):
            if pat in cu:
                return cols[i]
    return None

def _read_csv_chunks(fp: Path, usecols: list[str], sep: str, encoding: str):
    try:
        return pd.read_csv(
            fp,
            usecols=usecols,
```

```
            dtype=str,
            sep=sep,
            chunksize=SRI_CHUNK_ROWS,
            low_memory=True,
            encoding=encoding,
            on_bad_lines="skip",
        )
    except TypeError:
        return pd.read_csv(
            fp,
            usecols=usecols,
            dtype=str,
            sep=sep,
            chunksize=SRI_CHUNK_ROWS,
            low_memory=True,
            encoding=encoding,
            error_bad_lines=False,
            warn_bad_lines=True,
        )

def _read_sri_fantasia_chunks(fp: Path):
    """Iterador de chunks con columnas [NUMERO_RUC, NOMBRE_FANTASIA_COMERCIAL]␣
 ↪o equivalentes."""
    sep = _guess_sep(fp)
    encodings = ["utf-8", "latin1"]
    preferred = ["NUMERO_RUC", "NOMBRE_FANTASIA_COMERCIAL"]
    for enc in encodings:
        try:
            return _read_csv_chunks(fp, usecols=preferred, sep=sep,␣
 ↪encoding=enc)
        except Exception:
            pass

    # Detectar equivalentes por header
    try:
        cols = pd.read_csv(fp, nrows=0, sep=sep, encoding="latin1").columns.
 ↪tolist()
    except Exception as e:
        raise RuntimeError(f"No pude leer encabezado de {fp.name}: {e}") from e

    ruc_col = _pick_col(cols, patterns=["NUMERO_RUC", "RUC", "IDENTIFIC"])
    fant_col = _pick_col(cols, patterns=["NOMBRE_FANTASIA", "FANTASIA", "NOMBRE␣
 ↪COMERCIAL", "COMERCIAL"])
    if ruc_col is None or fant_col is None:
        raise RuntimeError(f"No pude detectar columnas RUC/FANTASIA en {fp.
 ↪name}. Columnas: {cols[:20]}")
```

```
    for enc in encodings:
        try:
            return _read_csv_chunks(fp, usecols=[ruc_col, fant_col], sep=sep,␣
 ↪encoding=enc)
        except Exception:
            pass

    raise RuntimeError(f"No pude leer {fp.name} con columnas detectadas.")

# --- Catálogo maestro sri_fantasia_best (deduplicado por nombre fantasía␣
 ↪normalizado) ---
count_map_f: dict[tuple[str, str], int] = {}  # (fantasia_norm, ruc) ->␣
 ↪frecuencia
n_rows_seen = 0
n_rows_kept = 0

for i, fp in enumerate(sri_files, start=1):
    print(f"[SRI/FANTASIA] ({i}/{len(sri_files)}) Leyendo: {fp.name}")
    try:
        chunk_iter = _read_sri_fantasia_chunks(fp)
    except Exception as e:
        print(f"[SRI/FANTASIA]   Saltado (no legible): {fp.name} -> {e}")
        continue

    for chunk in chunk_iter:
        if chunk is None or chunk.empty:
            continue
        n_rows_seen += int(len(chunk))

        cols = list(chunk.columns)
        ruc_col = _pick_col(cols, patterns=["NUMERO_RUC", "RUC", "IDENTIFIC"])␣
 ↪or cols[0]
        fant_col = _pick_col(cols, patterns=["NOMBRE_FANTASIA", "FANTASIA",␣
 ↪"COMERCIAL"])
        if fant_col is None:
            fant_col = cols[1] if len(cols) > 1 else None
        if fant_col is None:
            continue

        tmp = pd.DataFrame({
            "ruc_raw": chunk[ruc_col],
            "fantasia_raw": chunk[fant_col],
        })
        tmp["fantasia_raw"] = tmp["fantasia_raw"].astype("string")
        tmp = tmp[tmp["fantasia_raw"].notna() & (tmp["fantasia_raw"].str.
 ↪strip() != "")].copy()
```

```
        if tmp.empty:
            continue

        tmp["ruc"] = tmp["ruc_raw"].map(_normalize_ruc)
        tmp["fantasia_norm"] = tmp["fantasia_raw"].map(lambda x:␣
 ↪normalize_company_name(x))

        # Filtros mínimos
        tmp = tmp[tmp["ruc"].astype("string").str.len() == 13].copy()
        tmp = tmp[tmp["fantasia_norm"].str.contains(r"[A-Z]", regex=True,␣
 ↪na=False)].copy()
        tmp = tmp[tmp["fantasia_norm"].str.len() >= 3].copy()
        if tmp.empty:
            continue

        n_rows_kept += int(len(tmp))
        grp = tmp.groupby(["fantasia_norm", "ruc"]).size()
        for (name_norm, ruc), cnt in grp.items():
            key = (str(name_norm), str(ruc))
            count_map_f[key] = count_map_f.get(key, 0) + int(cnt)

print("[SRI/FANTASIA] Filas leídas (aprox):", n_rows_seen)
print("[SRI/FANTASIA] Filas retenidas tras filtros:", n_rows_kept)
print("[SRI/FANTASIA] Pares (fantasia_norm, ruc) únicos:", len(count_map_f))

if not count_map_f:
    raise ValueError("No se pudo construir catálogo SRI de fantasía␣
 ↪(count_map_f vacío).")

sri_fantasia_lookup = (
    pd.DataFrame([
        {"fantasia_name_norm": k[0], "RUC": k[1], "freq": v}
        for k, v in count_map_f.items()
    ])
    .sort_values(["fantasia_name_norm", "freq"], ascending=[True, False])
    .reset_index(drop=True)
)

sri_fantasia_best = (
    sri_fantasia_lookup
    .drop_duplicates(subset=["fantasia_name_norm"], keep="first")
    [["fantasia_name_norm", "RUC"]]
    .reset_index(drop=True)
)
print("[SRI/FANTASIA] sri_fantasia_best (nombres distinct):",␣
 ↪int(sri_fantasia_best.shape[0]))
```

```
# --- Fuzzy matching: LEADS huérfanas restantes -> SRI fantasía ---
try:
    from rapidfuzz import process, fuzz
except Exception as e:
    raise ImportError("Falta rapidfuzz. Instala con: pip install rapidfuzz")␣
 ↪from e

choice_names = sri_fantasia_best["fantasia_name_norm"].astype("string").
 ↪dropna().tolist()
rows = []
for _, r in unmatched_leads_fase3.iterrows():
    q = r.get("Company_norm", "")
    if q is None:
        continue
    q = str(q).strip()
    if not q:
        continue
    best = process.extractOne(q, choice_names, scorer=fuzz.token_set_ratio)
    if best is None:
        continue
    best_name, score, _ = best
    if float(score) >= float(UMBRAL_SRI):
        ruc = sri_fantasia_best.loc[sri_fantasia_best["fantasia_name_norm"] ==␣
 ↪best_name, "RUC"].iloc[0]
        rows.append({
            "Company_raw": r.get("Company_raw", ""),
            "Company_norm": q,
            "best_fantasia_name_norm": best_name,
            "score": float(score),
            "RUC": str(ruc),
        })

leads_fantasia_sugerido = pd.DataFrame(rows)
if leads_fantasia_sugerido.empty:
    leads_fantasia_sugerido = pd.DataFrame(columns=["Company_raw",␣
 ↪"Company_norm", "best_fantasia_name_norm", "score", "RUC"])
else:
    leads_fantasia_sugerido = leads_fantasia_sugerido.sort_values("score",␣
 ↪ascending=False).reset_index(drop=True)

print(f"[LEADS/FANTASIA] Umbral fuzzy SRI (UMBRAL_SRI): {UMBRAL_SRI}")
print(f"[LEADS/FANTASIA] Sugerencias fuzzy fantasía:␣
 ↪{int(leads_fantasia_sugerido.shape[0])}")
display(leads_fantasia_sugerido.head(20))
```

[38]:
```
# === 9.d Verificación RUC para HORAS vs Super (SCVS) ===
# Salida principal: `horas_ruc_exact` (match exacto) y `horas_ruc_sugerido` (match por similitud para no-matcheados)

import os
import pandas as pd
from pathlib import Path

if "horas_empresas_clean" not in globals():
    raise NameError("No existe `horas_empresas_clean`. Ejecuta primero la celda 9.b (limpieza HORAS).")

# Asegurar SUPER_DIR aunque esta celda se ejecute primero
if "SUPER_DIR" not in globals():
    _CANDIDATE_SUPER_DIRS = [
        Path("data_super_compañias"),
        Path("01_data_ingestion_enrichment") / "data_super_compañias",
    ]
    SUPER_DIR = next((p for p in _CANDIDATE_SUPER_DIRS if p.exists()), _CANDIDATE_SUPER_DIRS[0])

# Reusar el catálogo ya cargado (si existe); si no, cargarlo
if "super_best" not in globals():
    super_all, super_best = _load_super_companies(SUPER_DIR)

horas_ruc_exact = (
    horas_empresas_clean
    .merge(super_best, left_on="EMPRESA_norm", right_on="super_name_norm", how="left")
    .drop(columns=["super_name_norm"])
    .rename(columns={"super_ruc": "RUC"})
)

n_total = int(horas_ruc_exact.shape[0])
n_con_ruc = int(horas_ruc_exact["RUC"].notna().sum())
print(f"[HORAS] Total empresas distinct: {n_total}")
print(f"[HORAS] Con RUC (match exacto): {n_con_ruc} ({(n_con_ruc/max(n_total,1))*100:.1f}%)")

# Sugerencias fuzzy solo para los no-matcheados
# Por defecto bajamos a 80 para explorar cobertura (ajustable con SCVS_FUZZY_THRESHOLD)
UMBRAL_SIMILITUD = int(os.getenv("SCVS_FUZZY_THRESHOLD", "80"))
unmatched = horas_ruc_exact[horas_ruc_exact["RUC"].isna()].copy()

choices = super_best.copy()
choices["name_norm"] = choices["super_name_norm"]
```

```
choices = choices[["name_norm", "super_ruc"]].dropna().drop_duplicates().
 ↪reset_index(drop=True)
choice_names = choices["name_norm"].tolist()

rows = []
try:
    from rapidfuzz import process, fuzz
    for _, r in unmatched.iterrows():
        q = r["EMPRESA_norm"]
        if not q:
            continue
        best = process.extractOne(q, choice_names, scorer=fuzz.token_set_ratio)
        if best is None:
            continue
        best_name, score, _ = best
        if float(score) >= UMBRAL_SIMILITUD:
            ruc = choices.loc[choices["name_norm"] == best_name, "super_ruc"].
 ↪iloc[0]
            rows.append({
                "EMPRESA_raw": r["EMPRESA_raw"],
                "EMPRESA_norm": q,
                "best_super_name_norm": best_name,
                "score": float(score),
                "RUC": ruc,
            })
    engine = "rapidfuzz"
except Exception:
    from difflib import SequenceMatcher

    def _ratio(a, b):
        return SequenceMatcher(None, a, b).ratio() * 100

    for _, r in unmatched.iterrows():
        q = r["EMPRESA_norm"]
        if not q:
            continue
        best_name = None
        best_score = -1.0
        for cand in choice_names:
            sc = _ratio(q, cand)
            if sc > best_score:
                best_score = sc
                best_name = cand
        if best_name is not None and best_score >= UMBRAL_SIMILITUD:
            ruc = choices.loc[choices["name_norm"] == best_name, "super_ruc"].
 ↪iloc[0]
            rows.append({
```

```
                "EMPRESA_raw": r["EMPRESA_raw"],
                "EMPRESA_norm": q,
                "best_super_name_norm": best_name,
                "score": float(best_score),
                "RUC": ruc,
            })
    engine = "difflib"

horas_ruc_sugerido = pd.DataFrame(rows)
if horas_ruc_sugerido.empty:
    horas_ruc_sugerido = pd.DataFrame(columns=["EMPRESA_raw", "EMPRESA_norm",␣
 ↪"best_super_name_norm", "score", "RUC"])
else:
    horas_ruc_sugerido = horas_ruc_sugerido.sort_values("score",␣
 ↪ascending=False).reset_index(drop=True)

print(f"[HORAS] Motor fuzzy: {engine}")
print(f"[HORAS] Sugerencias fuzzy con score >= {UMBRAL_SIMILITUD}:␣
 ↪{int(horas_ruc_sugerido.shape[0])}")

display(horas_ruc_exact.head(20))
print("\n--- Sugerencias fuzzy (top 20) ---")
display(horas_ruc_sugerido.head(20))
```

```
[HORAS] Total empresas distinct: 119
[HORAS] Con RUC (match exacto): 18 (15.1%)
[HORAS] Motor fuzzy: rapidfuzz
[HORAS] Sugerencias fuzzy con score >= 80: 72
```

	EMPRESA_raw	EMPRESA_norm	RUC
0	3dpharma	3DPHARMA	NaN
1	Adium	ADIUM	NaN
2	Almexa	ALMEXA	NaN
3	Alper Seguros	ALPER SEGUROS	NaN
4	Arauco	ARAUCO	NaN
5	Aseguradora del Sur	ASEGURADORA SUR	NaN
6	Asesoría y Control	ASESORIA CONTROL	1792765617001
7	AutoShare	AUTOSHARE	NaN
8	AVIS	AVIS	1793119549001
9	BAC	BAC	NaN
10	Banco del Austro	BANCO AUSTRO	NaN
11	Banco Financiero	BANCO FINANCIERO	NaN
12	BANRED	BANRED	0991325026001
13	Bekaert	BEKAERT	NaN
14	Bemis	BEMIS	NaN
15	Bitso	BITSO	NaN
16	Bloomin Brands	BLOOMIN BRANDS	NaN
17	BMI	BMI	NaN

```
18          Boston BSCI       BOSTON BSCI            NaN
19                CEPSA             CEPSA  1790003388001


--- Sugerencias fuzzy (top 20) ---

              EMPRESA_raw            EMPRESA_norm                                 ␣
 ↪best_super_name_norm  score             RUC
0                      BAC                     BAC                                 ␣
 ↪ BAC MEDICAL BACME S  100.0  1793212358001
1      Aseguradora del Sur         ASEGURADORA SUR                                 ␣
 ↪ ASEGURADORA SUR C A  100.0  0190123626001
2       Corporación Maresa      CORPORACION MARESA                                 ␣
 ↪CORPORACION MARESA HOLDING S  100.0  1791397622001
3                    CORSAM                  CORSAM                              ␣
 ↪CORPORACION SAMBORONDON CORSAM  100.0  0992347430001
4                     Danec                   DANEC                              ␣
 ↪DANEC ENERGY DANERGY S  100.0  1793236059001
5                    CONSEP                  CONSEP       CONSTRUCCIONES        ␣
 ↪SERVICIOS PETROLEROS CONSEP S  100.0  2191774629001
6                 Comandato               COMANDATO                  COMANDATO    ␣
 ↪CUENCA SOCIEDAD ANONIMA  100.0  0190002098001
7                       BMI                     BMI                          BMI  ␣
 ↪ECUADOR SEGUROS VIDA  100.0  1791301692001
8                    Unacem                  UNACEM                              ␣
 ↪TRANSPORTES UNACEM UTR  100.0  1793056849001
9                    Xcaret                  XCARET                              ␣
 ↪            XCARET S  100.0  2490398352001
10                Sunshine                SUNSHINE                 NATURE S      ␣
 ↪SUNSHINE PRODUCTS ECUADOR  100.0  1791308751001
11                      SIC                     SIC             SERVICIOS        ␣
 ↪INTEGRALES CALIFICADOS SIC S  100.0  1891811405001
12           Tecniseguros            TECNISEGUROS    TECNISEGUROS AGENCIA        ␣
 ↪ASESORA PRODUCTORA SEGUROS  100.0  1790048772001
13   Seguros del Pichincha       SEGUROS PICHINCHA                               ␣
 ↪           PICHINCHA  100.0  1792893771001
14         Corporacion GPF         CORPORACION GPF                               ␣
 ↪CORPORACION GRUPO FYBECA GPF  100.0  1792287413001
15                   Corona                  CORONA                              ␣
 ↪CONSTRUCTORA CIVIL CORONA JF TR  100.0  1990928203001
16  Hospital Metropolitano  HOSPITAL METROPOLITANO                                ␣
 ↪HOSPITAL METROPOLITANO S  100.0  1793178138001
17  Ideal Alambrec Bekaert  IDEAL ALAMBREC BEKAERT                               ␣
 ↪      IDEAL ALAMBREC  100.0  1790050947001
18                    KPMG                    KPMG  CONSORCIO SALOMON SMITH      ␣
 ↪BARNEY HAGLER BAILLY K…  100.0  0000000000040
```

```
19                    FPA                    FPA              FPA CONSTRUCTORA␣
 ↪INMOBILIARIA S B I C  100.0  0195096309001
```

[47]:
```
# === 9.d.1 Verificación RUC en Padrón SRI para HORAS huérfanas ===
# Objetivo: segundo intento de match (fuzzy) para HORAS sin RUC (huérfanas)␣
 ↪contra el catálogo SRI ya cargado en 9.c.1.

import os
import pandas as pd

if "horas_ruc_exact" not in globals():
    raise NameError("No existe `horas_ruc_exact`. Ejecuta primero 9.d.")
if "sri_best" not in globals():
    raise NameError("No existe `sri_best`. Ejecuta primero 9.c.1 para cargar/
 ↪armar el catálogo SRI.")

UMBRAL_SRI = int(os.getenv("UMBRAL_SRI", "80"))

# Huérfanas: HORAS sin RUC tras SCVS exacto
unmatched_horas = horas_ruc_exact[horas_ruc_exact["RUC"].isna()].copy()
print("[HORAS/SRI] Huérfanas (sin RUC por SCVS exacto):", int(unmatched_horas.
 ↪shape[0]))

try:
    from rapidfuzz import process, fuzz
except Exception as e:
    raise ImportError("Falta rapidfuzz. Instala con: pip install rapidfuzz")␣
 ↪from e

choice_names = sri_best["sri_name_norm"].astype("string").dropna().tolist()
rows = []
for _, r in unmatched_horas.iterrows():
    q = r.get("EMPRESA_norm", "")
    if q is None:
        continue
    q = str(q).strip()
    if not q:
        continue
    best = process.extractOne(q, choice_names, scorer=fuzz.token_set_ratio)
    if best is None:
        continue
    best_name, score, _ = best
    if float(score) >= float(UMBRAL_SRI):
        ruc = sri_best.loc[sri_best["sri_name_norm"] == best_name, "RUC"].
 ↪iloc[0]
        rows.append({
            "EMPRESA_raw": r.get("EMPRESA_raw", ""),
```

```
            "EMPRESA_norm": q,
            "best_sri_name_norm": best_name,
            "score": float(score),
            "RUC": str(ruc),
        })

horas_sri_sugerido = pd.DataFrame(rows)
if horas_sri_sugerido.empty:
    horas_sri_sugerido = pd.DataFrame(columns=["EMPRESA_raw", "EMPRESA_norm",␣
 ↪"best_sri_name_norm", "score", "RUC"])
else:
    horas_sri_sugerido = horas_sri_sugerido.sort_values("score",␣
 ↪ascending=False).reset_index(drop=True)

print(f"[HORAS/SRI] Umbral fuzzy SRI (UMBRAL_SRI): {UMBRAL_SRI}")
print(f"[HORAS/SRI] Sugerencias fuzzy SRI: {int(horas_sri_sugerido.shape[0])}")
display(horas_sri_sugerido.head(20))
```

```
[HORAS/SRI] Huérfanas (sin RUC por SCVS exacto): 101
[HORAS/SRI] Umbral fuzzy SRI (UMBRAL_SRI): 80
[HORAS/SRI] Sugerencias fuzzy SRI: 10
```

	EMPRESA_raw	EMPRESA_norm	best_sri_name_norm	score	RUC
0	Maderera	MADERERA	1791259548001 MADERERA GUACHALA	100.000000	1791259548001
1	Ministerio Salud Pública	MINISTERIO SALUD PUBLICA	1793218318001 SINDICATO NACIONAL TRABAJADORES …	100.000000	1793218318001
2	El Sabor	SABOR	0992813997001 D FINO AROMA SABOR	100.000000	0992813997001
3	Municipio de Quito	MUNICIPIO QUITO	1792118689001 ASOCIACION EMPLEADOS MUNICIPIO D…	100.000000	1792118689001
4	PUCE	PUCE	1792043964001 FONDO INVERSON SOCIAL PROFESORES…	100.000000	1792043964001
5	Reybanpac	REYBANPAC	0990326606001 REYBANPAC	100.000000	0990326606001
6	Tecniseguros	TECNISEGUROS	0990705836001 TECNISEGUROS GUAYAQUIL	100.000000	0990705836001
7	Telefonica CR	TELEFONICA CR	0991408479001 EMPRESA INGENIERIA TELEFONICA EL…	86.956522	0991408479001
8	Telefónica EC	TELEFONICA EC	0991408479001 EMPRESA INGENIERIA TELEFONICA EL…	86.956522	0991408479001
9	Corporacion GPF	CORPORACION GPF	0190329798001 CORPORACION SERVICIOS ESPECIALIZ…	84.615385	0190329798001

[ ]:
```
# === 9.d.2 Verificación RUC por NOMBRE_FANTASIA_COMERCIAL para HORAS ===
```

```
# Objetivo: tercer intento de match (fuzzy) usando el nombre comercial/fantasía␣
 ↪del SRI, solo para huérfanas que siguen sin match por Razón Social.
# Reutilización: NO re-lee archivos del SRI; usa `sri_fantasia_best` construido␣
 ↪en 9.c.2.

import os
import pandas as pd

# --- Requisitos de entrada ---
if "unmatched_horas" not in globals():
    raise NameError("No existe `unmatched_horas`. Ejecuta primero 9.d.1.")
if "horas_sri_sugerido" not in globals():
    raise NameError("No existe `horas_sri_sugerido`. Ejecuta primero 9.d.1.")
if "sri_fantasia_best" not in globals():
    raise NameError("No existe `sri_fantasia_best`. Ejecuta primero 9.c.2.")

UMBRAL_SRI = int(os.getenv("UMBRAL_SRI", "80"))

# --- Huérfanas restantes (fase 3): excluir las que ya tuvieron sugerencia por␣
 ↪Razón Social ---
matched_raw = set(
    horas_sri_sugerido["EMPRESA_raw"].astype("string").fillna("").map(lambda x:␣
 ↪x.strip()).tolist()
    if not horas_sri_sugerido.empty and "EMPRESA_raw" in horas_sri_sugerido.
 ↪columns
    else []
)
unmatched_horas_fase3 = unmatched_horas.copy()
if "EMPRESA_raw" in unmatched_horas_fase3.columns:
    unmatched_horas_fase3 =␣
 ↪unmatched_horas_fase3[~unmatched_horas_fase3["EMPRESA_raw"].astype("string").
 ↪fillna("").map(lambda x: x.strip()).isin(matched_raw)].copy()
print("[HORAS/FANTASIA] Huérfanas restantes para fase 3:",␣
 ↪int(unmatched_horas_fase3.shape[0]))

# --- Fuzzy matching: HORAS huérfanas restantes -> SRI fantasía ---
try:
    from rapidfuzz import process, fuzz
except Exception as e:
    raise ImportError("Falta rapidfuzz. Instala con: pip install rapidfuzz")␣
 ↪from e

choice_names = sri_fantasia_best["fantasia_name_norm"].astype("string").
 ↪dropna().tolist()
rows = []
for _, r in unmatched_horas_fase3.iterrows():
```

```
    q = r.get("EMPRESA_norm", "")
    if q is None:
        continue
    q = str(q).strip()
    if not q:
        continue
    best = process.extractOne(q, choice_names, scorer=fuzz.token_set_ratio)
    if best is None:
        continue
    best_name, score, _ = best
    if float(score) >= float(UMBRAL_SRI):
        ruc = sri_fantasia_best.loc[sri_fantasia_best["fantasia_name_norm"] ==␣
 ↪best_name, "RUC"].iloc[0]
        rows.append({
            "EMPRESA_raw": r.get("EMPRESA_raw", ""),
            "EMPRESA_norm": q,
            "best_fantasia_name_norm": best_name,
            "score": float(score),
            "RUC": str(ruc),
        })

horas_fantasia_sugerido = pd.DataFrame(rows)
if horas_fantasia_sugerido.empty:
    horas_fantasia_sugerido = pd.DataFrame(columns=["EMPRESA_raw",␣
 ↪"EMPRESA_norm", "best_fantasia_name_norm", "score", "RUC"])
else:
    horas_fantasia_sugerido = horas_fantasia_sugerido.sort_values("score",␣
 ↪ascending=False).reset_index(drop=True)

print(f"[HORAS/FANTASIA] Umbral fuzzy SRI (UMBRAL_SRI): {UMBRAL_SRI}")
print(f"[HORAS/FANTASIA] Sugerencias fuzzy fantasía:␣
 ↪{int(horas_fantasia_sugerido.shape[0])}")
display(horas_fantasia_sugerido.head(20))
```

[ ]:
```
# === 9.e (Obligatorio si vas a usar 9.f) Preparar auditoría LLM para␣
 ↪resultados fuzzy ===
# Objetivo: mandar a ChatGPT/OpenAI las sugerencias fuzzy provenientes de:
# - SCVS/SUPER (LEADS y HORAS)
# - SRI por Razón Social (LEADS y HORAS)
# - SRI por Nombre Fantasía/Comercial (LEADS y HORAS)
# para validación.

import os
import json
import pandas as pd

# Requeridos (SUPER)
```

```
if "leads_ruc_sugerido" not in globals() or "horas_ruc_sugerido" not in␣
 ↪globals():
    raise NameError("Faltan `leads_ruc_sugerido` u `horas_ruc_sugerido`.␣
 ↪Ejecuta primero 9.c y 9.d.")

# (SRI) pueden no existir si no corriste 9.c.1 / 9.d.1 / 9.c.2 / 9.d.2
leads_sri_sugerido = globals().get("leads_sri_sugerido")
horas_sri_sugerido = globals().get("horas_sri_sugerido")
leads_fantasia_sugerido = globals().get("leads_fantasia_sugerido")
horas_fantasia_sugerido = globals().get("horas_fantasia_sugerido")

# Para ahorrar tokens/costo: recorta textos muy largos
MAX_NAME_CHARS = int(os.getenv("AUDIT_MAX_NAME_CHARS", "160"))
def _clip_text(x, n: int = MAX_NAME_CHARS) -> str:
    if x is None or (isinstance(x, float) and pd.isna(x)):
        return ""
    s = str(x)
    return s if len(s) <= n else (s[:n] + "…")

# Umbrales (por defecto 80 ambos, pero se permiten configurar por separado)
AUDIT_THRESHOLD_SUPER = int(os.getenv("SCVS_FUZZY_THRESHOLD", str(globals().
 ↪get("UMBRAL_SIMILITUD", 80))))
AUDIT_THRESHOLD_SRI = int(os.getenv("UMBRAL_SRI", "80"))

# Prompt de auditoría (compacto). Regla clave: NO inventar; decidir solo con␣
 ↪los textos entregados.
LLM_JUDGE_PROMPT = """Eres auditor de matching de empresas (Ecuador).
Entrada: JSON {cases:[{id,source_raw,candidate_norm,ruc,score}]}.
Decide si el match es plausible SOLO con esos textos (sin conocimiento externo).

Reglas:
- Si ruc no tiene 13 dígitos => verdict=incorrect (confidence alta).
- Si comparten tokens distintivos y no hay conflicto => correct.
- Si es genérico, corto o evidencia insuficiente => uncertain.

Salida: JSON con clave results = lista de objetos {id, verdict, confidence,␣
 ↪reason}.
confidence en [0,1]. reason: 1 frase corta (<= 80 chars), sin saltos de línea y␣
 ↪SIN comillas dobles (usa comillas simples si hace falta)."""

def _build_all_cases(df_sugerido: pd.DataFrame, source_label: str, threshold:␣
 ↪int) -> list:
    """Mapea cualquier DF de sugerencias (SUPER o SRI) a la estructura de␣
 ↪auditoría."""
    if df_sugerido is None or getattr(df_sugerido, "empty", True):
        return []
```

```
    df = df_sugerido.copy()
    if "score" in df.columns:
        df = df[df["score"].astype(float) >= float(threshold)].copy()
    if df.empty:
        return []
    df = df.sort_values("score", ascending=False).reset_index(drop=True)

    # Detectar columnas comunes
    source_raw_col = "Company_raw" if "Company_raw" in df.columns else␣
↪("EMPRESA_raw" if "EMPRESA_raw" in df.columns else None)
    candidate_col = None
    for c in ["best_super_name_norm", "best_sri_name_norm",␣
↪"best_fantasia_name_norm", "candidate_norm"]:
        if c in df.columns:
            candidate_col = c
            break
    ruc_col = "RUC" if "RUC" in df.columns else ("ruc" if "ruc" in df.columns␣
↪else None)

    cases = []
    for i, r in df.iterrows():
        case_id = f"{source_label}-{i}"
        source_raw = r.get(source_raw_col, "") if source_raw_col else ""
        candidate_norm = r.get(candidate_col, "") if candidate_col else ""
        ruc = r.get(ruc_col, "") if ruc_col else ""
        cases.append({
            "id": case_id,
            "source_label": source_label,
            "source_raw": _clip_text(source_raw),
            "candidate_norm": _clip_text(candidate_norm),
            "score": None if pd.isna(r.get("score")) else float(r.get("score")),
            "ruc": "" if pd.isna(ruc) else str(ruc),
        })
    return cases

audit_cases = []
audit_cases += _build_all_cases(leads_ruc_sugerido, "LEADS_SUPER",␣
↪AUDIT_THRESHOLD_SUPER)
audit_cases += _build_all_cases(horas_ruc_sugerido, "HORAS_SUPER",␣
↪AUDIT_THRESHOLD_SUPER)
audit_cases += _build_all_cases(leads_sri_sugerido, "LEADS_SRI_RAZON",␣
↪AUDIT_THRESHOLD_SRI)
audit_cases += _build_all_cases(horas_sri_sugerido, "HORAS_SRI_RAZON",␣
↪AUDIT_THRESHOLD_SRI)
audit_cases += _build_all_cases(leads_fantasia_sugerido, "LEADS_SRI_FANTASIA",␣
↪AUDIT_THRESHOLD_SRI)
```

```
audit_cases += _build_all_cases(horas_fantasia_sugerido, "HORAS_SRI_FANTASIA",␣
 ↪AUDIT_THRESHOLD_SRI)

# Métricas claras
n_leads_super = int(getattr(leads_ruc_sugerido, "shape", [0])[0])
n_horas_super = int(getattr(horas_ruc_sugerido, "shape", [0])[0])
n_leads_sri_razon = int(getattr(leads_sri_sugerido, "shape", [0])[0]) if␣
 ↪leads_sri_sugerido is not None else 0
n_horas_sri_razon = int(getattr(horas_sri_sugerido, "shape", [0])[0]) if␣
 ↪horas_sri_sugerido is not None else 0
n_leads_sri_f = int(getattr(leads_fantasia_sugerido, "shape", [0])[0]) if␣
 ↪leads_fantasia_sugerido is not None else 0
n_horas_sri_f = int(getattr(horas_fantasia_sugerido, "shape", [0])[0]) if␣
 ↪horas_fantasia_sugerido is not None else 0

audit_df = pd.DataFrame(audit_cases)
counts = audit_df.groupby("source_label").size().to_dict() if not audit_df.
 ↪empty else {}

print("Umbral SUPER (SCVS_FUZZY_THRESHOLD):", AUDIT_THRESHOLD_SUPER)
print("Umbral SRI (UMBRAL_SRI):", AUDIT_THRESHOLD_SRI)
print("Sugerencias fuzzy disponibles:",
      f"LEADS_SUPER={n_leads_super}", ",",
      f"HORAS_SUPER={n_horas_super}", ",",
      f"LEADS_SRI_RAZON={n_leads_sri_razon}", ",",
      f"HORAS_SRI_RAZON={n_horas_sri_razon}", ",",
      f"LEADS_SRI_FANTASIA={n_leads_sri_f}", ",",
      f"HORAS_SRI_FANTASIA={n_horas_sri_f}")
print("Casos a auditar (armados) total:", len(audit_cases))
print("Casos a auditar por fuente:", counts)
print("Ejemplo (1 caso):")
print(json.dumps(audit_cases[:1], ensure_ascii=False, indent=2))
```

```
Umbral SUPER (SCVS_FUZZY_THRESHOLD): 80
Umbral SRI (UMBRAL_SRI): 80
Sugerencias fuzzy disponibles: LEADS_SUPER=181 , HORAS_SUPER=72 , LEADS_SRI=12 ,
HORAS_SRI=10
Casos a auditar (armados) total: 275
Casos a auditar por fuente: {'HORAS_SRI': 10, 'HORAS_SUPER': 72, 'LEADS_SRI':
12, 'LEADS_SUPER': 181}
Ejemplo (1 caso):
[
  {
    "id": "LEADS_SUPER-0",
    "source_label": "LEADS_SUPER",
    "source_raw": "ABBOTT",
    "candidate_norm": "ABBOTT LABORATORIOS ECUADOR",
```

```
    "score": 100.0,
    "ruc": "0990000670001"
  }
]
```

[49]:
```
# === 9.f (Opcional) Llamar API (ChatGPT/OpenAI) para auditar matches fuzzy ===
# Este paso manda TODOS los casos de `audit_cases` (celda 24 / 9.e) a la API,␣
 ↪en chunks para evitar truncados.

import os
import json
import urllib.request
import urllib.error
import pandas as pd

if "audit_cases" not in globals():
    raise NameError("No existe `audit_cases`. Ejecuta primero la celda 24 (9.e).
 ↪")

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
if not OPENAI_API_KEY:
    try:
        import getpass
        OPENAI_API_KEY = getpass.getpass("Ingresa OPENAI_API_KEY (entrada␣
 ↪oculta, no se guarda): ").strip()
    except Exception:
        OPENAI_API_KEY = None

if not OPENAI_API_KEY:
    raise EnvironmentError(
        "No se encontró `OPENAI_API_KEY`. "
        "Configúrala como variable de entorno o ingrésala cuando se solicite."
    )

OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")
OPENAI_URL = os.getenv("OPENAI_URL", "https://api.openai.com/v1/chat/
 ↪completions")


def _extract_json_object(text: str) -> str:
    if text is None:
        return ""
    s = str(text).strip()
    if s.startswith("```"):
        s = s.split("\n", 1)[1]
        s = s.rsplit("```", 1)[0].strip()
    i = s.find("{")
```

```
    j = s.rfind("}")
    return s[i:j+1] if i != -1 and j != -1 and j > i else s


def _call_openai_judge(cases_chunk: list) -> dict:
    payload = {
        "model": OPENAI_MODEL,
        "temperature": 0,
        # salida por chunk (ajústalo si ves truncado)
        "max_tokens": int(os.getenv("OPENAI_MAX_TOKENS", "2000")),
        "response_format": {"type": "json_object"},
        "messages": [
            {"role": "system", "content": LLM_JUDGE_PROMPT},
            {"role": "user", "content": json.dumps({"cases": cases_chunk},␣
 ↪ensure_ascii=False)},
        ],
    }

    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        OPENAI_URL,
        data=data,
        headers={"Content-Type": "application/json", "Authorization": f"Bearer␣
 ↪{OPENAI_API_KEY}"},
        method="POST",
    )

    try:
        with urllib.request.urlopen(req, timeout=300) as resp:
            raw = resp.read().decode("utf-8")
    except urllib.error.HTTPError as e:
        err = e.read().decode("utf-8", errors="ignore")
        raise RuntimeError(f"HTTPError {e.code}: {err}")

    out = json.loads(raw)
    content = out["choices"][0]["message"]["content"]
    content = _extract_json_object(content)
    return json.loads(content)


def _judge_in_chunks(cases_all: list) -> dict:
    if not cases_all:
        return {"results": []}

    chunk_size = int(os.getenv("AUDIT_CHUNK_SIZE", "10"))
    min_chunk = int(os.getenv("AUDIT_MIN_CHUNK", "2"))
    results_all = []
```

```
    i = 0
    while i < len(cases_all):
        cur = cases_all[i:i+chunk_size]
        try:
            out = _call_openai_judge(cur)
            res = out.get("results", [])
            if not isinstance(res, list):
                raise ValueError("Respuesta sin results-list")
            results_all.extend(res)
            i += chunk_size
        except (json.JSONDecodeError, ValueError) as e:
            if chunk_size <= min_chunk:
                raise RuntimeError(f"No pude parsear JSON con␣
 ↪chunk_size={chunk_size}. Error: {e}") from e
            chunk_size = max(min_chunk, chunk_size // 2)
            continue

    return {"results": results_all}


print("Casos a enviar a IA:", len(audit_cases))
judge_json = _judge_in_chunks(list(audit_cases))
results = judge_json.get("results", [])

judge_df = pd.DataFrame(results)
if judge_df.empty:
    raise ValueError("La API respondió, pero no devolvió `results`. Revisa el␣
 ↪JSON retornado.")

cases_df = pd.DataFrame(audit_cases)
out_df = cases_df.merge(judge_df, on="id", how="left")

print("Resultados auditados:", len(out_df))
print(out_df["verdict"].value_counts(dropna=False))

cols = ["source_label", "source_raw", "candidate_norm", "score", "ruc",␣
 ↪"verdict", "confidence", "reason"]
cols = [c for c in cols if c in out_df.columns]
display(out_df.sort_values(["source_label", "score"], ascending=[True,␣
 ↪False])[cols].head(120))
```

```
Casos a enviar a IA: 275
Resultados auditados: 275
verdict
correct      138
uncertain    129
```

```
incorrect      8
Name: count, dtype: int64
```

```
    source_label                source_raw                                          ␣
↪candidate_norm  score            ruc    verdict  confidence                       ␣
↪                          reason
265    HORAS_SRI                   Maderera                     1791259548001␣
↪MADERERA GUACHALA  100.0  1791259548001    correct        0.95         Exact␣
↪token maderera present in both names
266    HORAS_SRI  Ministerio Salud Pública  1793218318001 SINDICATO NACIONAL␣
↪TRABAJADORES …  100.0  1793218318001    correct        0.95  Shared␣
↪distinctive tokens ministerio salud pub…
267    HORAS_SRI                   El Sabor                    0992813997001 D␣
↪FINO AROMA SABOR  100.0  0992813997001  uncertain        0.60  Token sabor␣
↪present but candidate name is generic
268    HORAS_SRI        Municipio de Quito  1792118689001 ASOCIACION EMPLEADOS␣
↪MUNICIPIO D…  100.0  1792118689001    correct        0.95   Shared tokens␣
↪municipio and quito indicate match
269    HORAS_SRI                      PUCE  1792043964001 FONDO INVERSON SOCIAL␣
↪PROFESORES…  100.0  1792043964001    correct        0.90                    ␣
↪Token puce present in both names
..           …                         …                                          ␣
↪     …    …            …        …         …                                    ␣
↪           …
21   LEADS_SUPER                 COCA-COLA                                   COCA␣
↪COLA ECUADOR  100.0  1791324404001    correct        0.90       RUC valid and␣
↪tokens COCA COLA match clearly
22   LEADS_SUPER           CLG TRANSPORTES                                        ␣
↪         CLG  100.0  0993044008001    correct        0.80  RUC valid and CLG␣
↪token matches source and can…
23   LEADS_SUPER         COLGATE PALMOLIVE  COLGATE PALMOLIVE ECUADOR SOCIEDAD␣
↪ANONIMA IND…  100.0  1790337979001    correct        0.90  RUC valid and␣
↪COLGATE PALMOLIVE tokens match c…
24   LEADS_SUPER                     COMEX  ASOCIADOS EN SOLUCIONES␣
↪EMPRESARIALES COMEX AS…  100.0  0993290599001  uncertain        0.50  COMEX is␣
↪generic and candidate has many extra …
25   LEADS_SUPER              OFFICE DEPOT                                        ␣
↪       DEPOT  100.0  0991307281001  uncertain        0.50  DEPOT is generic␣
↪and short compared to OFFICE …

[120 rows x 8 columns]
```

[50]:
```
# === Export final a CSV (salida de la última celda) ===
# Esta celda guarda a disco el DataFrame final (por defecto: `out_df`) para que␣
↪lo puedas descargar desde VS Code.

from pathlib import Path
```

```
from datetime import datetime
import pandas as pd

# 1) Detectar el DataFrame final (prioridad: out_df)
df_final = None
df_name = None

if "out_df" in globals() and isinstance(globals().get("out_df"), pd.DataFrame):
    df_final = globals()["out_df"]
    df_name = "out_df"
elif "audit_df" in globals() and isinstance(globals().get("audit_df"), pd.
 ↪DataFrame):
    df_final = globals()["audit_df"]
    df_name = "audit_df"
else:
    # fallback: intenta con el último df "razonable" en memoria
    for cand in ["leads_sri_sugerido", "horas_sri_sugerido",␣
 ↪"leads_ruc_sugerido", "horas_ruc_sugerido"]:
        if cand in globals() and isinstance(globals().get(cand), pd.DataFrame):
            df_final = globals()[cand]
            df_name = cand
            break

if df_final is None:
    raise NameError("No encontré un DataFrame final para exportar (esperaba␣
 ↪`out_df`). Ejecuta la última celda y vuelve a intentar.")
if df_final.empty:
    raise ValueError(f"El DataFrame `{df_name}` está vacío; no exporto un CSV␣
 ↪vacío.")

# 2) Ruta de salida robusta (si el cwd ya es 01_data_ingestion_enrichment, usa␣
 ↪outputs/)
OUT_DIR = Path("outputs") if Path.cwd().name == "01_data_ingestion_enrichment"␣
 ↪else (Path("01_data_ingestion_enrichment") / "outputs")
OUT_DIR.mkdir(parents=True, exist_ok=True)

stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
out_path = OUT_DIR / f"{df_name}_export_{stamp}.csv"

# 3) Guardar
df_final.to_csv(out_path, index=False, encoding="utf-8-sig")
print(f"CSV exportado desde `{df_name}` -> {out_path.resolve()}")
print("Filas:", len(df_final), "| Columnas:", df_final.shape[1])
```

```
CSV exportado desde `out_df` -> E:\TESIS MAESTRIA\Desarrollo_clustering_maestria
\01_data_ingestion_enrichment\outputs\out_df_export_20260412_163051.csv
Filas: 275 | Columnas: 9
```